

Student Fee Management System

Session 2024-2028

A project submitted in partial fulfillment of the degree of

BS in Computer Science



Department of Computer Science

Faculty of Computer Science & Information Technology

The Superior University, Lahore

Spring 2026

Type (Nature of project)	[☒] Development [☐] Research [☐] R&D			
Area of specialization	Software Engineering			
FYP ID				
Project Group Members				
Sr.#	Reg. #	Student Name	Email ID	*Signature
(i)	SU92-BSCSM-F24-020	M.Furqan	furqan209@gmail.com	
(ii)	SU92-BSCSM-F24-016	Hafiz Abdul Wahab	AbdulWahab23900@gmail.com	
(iii)				

*The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to work of others

Plagiarism Free Certificate

This is to certify that, I M.Furqan S/D of Muhammad Zubair, group leader of FYP under registration no SU92-BSCSM-F24-020 at Software Engineering Department, The Superior College, Lahore. I declare that my FYP report is checked by my supervisor.

Date: 12-5-2026 Name of Group Leader: M.Furqan Signature: _____

Name of Supervisor: Mrs. Humaira Co-Supervisor: Mr. XYZ

Designation: Lecturer Designation: Associate Professor

Signature: _____ Signature: _____

HoD: Dr. Muhammad Azam

Signature: _____

Project Report

Student Fee Management System

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature
M.Furqan	1.0	12-Feb-2026	Original Draft	
M.Furqan	1.1	13-March-2026	Updated Literature Review	
M.Furqan	1.2	6-5-2026	Full Project Complete	

APPROVAL

PROJECT SUPERVISOR

Comments: _____

Name: _____

Date: _____ Signature: _____

PROJECT MANAGER

Comments: _____

Date: _____ Signature: _____

HEAD OF THE DEPARTMENT

Comments: _____

Date: _____ Signature: _____

Dedication

This work is dedicated to our parents, teachers, and everyone who supported us throughout our academic journey.

Acknowledgements

We are sincerely thankful to our respected supervisor for his guidance, support, and encouragement throughout the development of this project. We are also grateful to our teachers, parents, and friends for their continuous support and motivation.

Executive Summary

The Student Fee Management System is a comprehensive software solution designed to automate and streamline fee-related operations in educational institutions. This project addresses the critical need for an efficient, secure, and reliable alternative to manual fee management methods, which are often prone to errors, data redundancy, and operational delays.

The system provides role-based access control for two primary user types: Admin and Accounts Officer. The Admin has full system privileges including user management, student record management, class and semester configuration, report generation, and data backup and restore operations. The Accounts Officer handles daily fee-related tasks including defining fee structures, processing full or partial payments, calculating dues and fines, and generating receipts and financial reports.

The system comprises eight major functional modules. The User Authentication module ensures secure access through username and password validation with role-based permissions. The Student Management module allows adding, updating, viewing, and searching student records. The Class and Semester Management module enables creation of academic sessions and assignment of students to specific classes. The Fee Structure Management module supports defining various fee types such as tuition, laboratory, and library fees with semester-wise amounts and due dates.

The Fee Payment Processing module records full or partial payments, automatically generates payment dates, and maintains complete payment history for each student. The Dues and Fine Management module calculates remaining dues, applies late fee fines automatically based on overdue payments, and identifies defaulters. The Reports and Receipts module generates individual fee receipts, paid students reports, defaulters lists, and total collection summaries. The

Data Storage and Backup module ensures secure data persistence with manual backup and restore capabilities to prevent data loss.

The project follows a structured Waterfall development methodology with clearly defined phases including requirement analysis, system design, implementation, testing, deployment, and documentation. The system is developed using C++ with file handling for data storage, targeting the Windows operating system environment. A two-week development timeline is planned with dedicated phases for each major activity.

Comprehensive testing validates system functionality, reliability, and performance. Use case testing confirms correct behavior for normal and exceptional scenarios. Equivalence partitioning and boundary value analysis ensure proper handling of input ranges. Unit testing verifies individual modules function correctly in isolation. Integration testing confirms seamless interaction between integrated components. Performance and stress testing demonstrate the system's ability to handle expected loads and recover from extreme conditions.

The Student Fee Management System successfully achieves its objectives of reducing manual workload, minimizing errors, ensuring accurate financial record keeping, and providing quick access to critical information. The system offers educational institutions a cost-effective, user-friendly, and reliable solution that significantly improves efficiency and transparency in fee management operations compared to traditional manual methods

Table of Contents

Dedication	v
Acknowledgements.....	vi
Executive Summary.....	vii
Table of Contents.....	ix
Chapter 1.....	1
Introduction	1
1.1. Background.....	2
1.2. Motivations and Challenges.....	2
1.3. Goals and Objectives.....	3
1.4. Literature Review/Existing Solutions	3
1.5. Gap Analysis	4
1.6. Proposed Solution	4
1.7. Project Plan	4
1.7.1. Work Breakdown Structure.....	8
1.7.2. Roles & Responsibility Matrix.....	9
1.7.3. Gantt Chart	10
1.8. Report Outline.....	10
Chapter 2.....	13
Software Requirement Specifications	13
2.1. Introduction.....	14
2.1.1. Purpose.....	14
2.1.2. Document Conventions	15
2.1.3. Intended Audience and Reading Suggestions	15
2.1.4. Product Scope.....	16

2.1.5. References	16
2.2. Overall Description	17
2.2.1. Product Perspective.....	17
2.2.2. Product Functions.....	17
2.2.3. User Classes and Characteristics	18
2.2.4. Operating Environment	19
2.2.5. Design and Implementation Constraints.....	19
2.2.6. User Documentation	20
2.2.7. Assumptions and Dependencies	20
2.3. External Interface Requirements	21
2.3.1. User Interfaces.....	21
2.3.2. Hardware Interfaces	22
2.3.3. Software Interfaces	22
2.3.4. Communications Interfaces.....	24
2.4. System Features	24
2.4.1. System Feature 1	26
2.4.1.1. Description and Priority	26
2.4.1.2. Stimulus/Response Sequences	27
2.4.1.3. Functional Requirements.....	27
2.4.2. System Feature 2	27
2.4.2.1. Description and Priority	28
2.4.2.2. Stimulus/Response Sequences	28
2.4.2.3. Functional Requirements.....	28
2.4.3. System Feature 3 (and so on)	29
2.5. Other Nonfunctional Requirements	35
2.5.1. Performance Requirements	35
2.5.2. Safety Requirements	36
2.5.3. Security Requirements	36
2.5.4. Software Quality Attributes.....	37
2.5.5. Business Rules.....	38
2.6. Other Requirements.....	38
Chapter 3.....	39
Use Case Analysis	40
3.1. Use Case Model.....	43
3.2. Use Case Descriptions	44
Chapter 4.....	51
System Design	51
4.1. Architecture Diagram	52
4.2. Domain Model.....	53
4.3. Entity Relationship Diagram with data dictionary	54
4.4. Class Diagram	58
4.5. Sequence / Collaboration Diagram	60
4.6. Operation contracts	64
4.7. Activity Diagram	69

4.8. State Transition Diagram.....	75
4.9. Component Diagram	79
4.11. Data Flow diagram [only if structured approach is used - Level 0 and 1]	80
Chapter 6.....	83
Testing and Evaluation.....	83
6.1. Use Case Testing.....	84
6.2. Equivalence partitioning	85
6.3. Boundary value analysis.....	86
6.4. Data flow testing	88
6.5. Unit testing.....	89
6.6. Integration testing.....	90
6.7. Performance testing.....	91
6.8. Stress Testing	93

List of Figures

1.1.	Work Breakdown Structure of Student Fee Management System	9
1.2.	Gantt Chart of Student Fee Management System	10
1.3.	Empathy Map of Student Fee Management System	12
3.1	Use Case Diagram of Student Fee Management System	44
4.1	Architecture Diagram of Student Fee Management System	52
4.2	Domain Model Diagram of Student Fee Management System	53
4.3	Entity Relationship Diagram of Student Fee Management System	54
4.4	Class Diagram of Student Fee Management System	59
4.5	Sequence Diagrams of Student Fee Management System	60-64
4.6	Activity Diagrams of Student Fee Management System	69-74
4.7	State Transition Diagrams of Student Fee Management System	75-78
4.8	Component Diagrams of Student Fee Management System	79-80
4.9	Data Flow Diagrams of Student Fee Management System	80-82

List of Tables

1.1	Project Timeline of Student Fee Management System	7
1.2	Roles & Responsibility Matrix of Student Fee Management System	10
4.1	Data Dictionary of Student Fee Management System	55-58
4.2	Operation Contracts of Student Fee Management System	64-69
6.1	Use Case Testing of Student Fee Management System	84-85
6.2	Equivalence partitioning Testing of Student Fee Management System	85-86
6.3	Boundary Testing of Student Fee Management System	86-87
6.4	Data Flow Testing of Student Fee Management System	88-89
6.5	Unit testing of Student Fee Management System	89-90
6.6	Integration Testing of Student Fee Management System	90-91
6.7	Performance Testing of Student Fee Management System	91-92
6.8	Stress Testing of Student Fee Management System	93-94

Chapter 1

Introduction

Chapter 1: Introduction

This chapter provides an overview of the Student Fee Management System and explains the purpose of developing this project. It highlights the importance of automating fee-related activities in educational institutions to reduce manual workload and errors. The chapter also introduces the basic concept of managing student records, fee structures, and payment processes through a computerized system. Additionally, it outlines how the system ensures secure access, accurate record keeping, and efficient fee control.

1.1. Background

In many educational institutions, fee management is still handled manually, which often leads to errors, data redundancy, and delays in record maintenance. Managing student information, fee structures, and payment records becomes difficult as the number of students increases. The lack of a centralized and secure system also makes it challenging to track dues, fines, and defaulters accurately. To overcome these issues, a computerized Student Fee Management System is required to streamline fee-related operations. This project aims to provide an efficient, secure, and reliable solution for managing student fees and financial records.

1.2. Motivations and Challenges

The main motivation behind developing the Student Fee Management System is to simplify and automate the complex process of fee handling in educational institutions. Manual systems are time-consuming and prone to human errors, which can affect financial accuracy and transparency. This system is motivated by the need for quick access to student fee records, accurate calculation of dues, and timely generation of reports.

However, several challenges are involved in developing such a system. Ensuring data security and role-based access control is a major concern. Designing an efficient structure for managing large amounts of student and payment data is also challenging. Additionally, maintaining data integrity, handling partial payments, and implementing a reliable backup and recovery mechanism require careful planning and implementation.

1.3. Goals and Objectives

The primary goal of the Student Fee Management System is to provide an efficient and automated solution for managing student fee records in educational institutions. The system aims to reduce manual work, minimize errors, and ensure accurate financial record keeping.

The main objectives of this project include maintaining complete student and fee information, enabling secure user authentication, and facilitating easy fee payment processing. The system also aims to calculate dues and fines automatically, generate detailed reports and receipts, and ensure data safety through proper storage and backup mechanisms. Overall, the project seeks to improve efficiency, transparency, and reliability in fee management operations

1.4. Literature Review/Existing Solutions

Several existing solutions and software applications aim to manage student information and financial records in educational institutions. Traditional systems often rely on manual record books or basic spreadsheet tools, which are prone to human error and make it difficult to retrieve or update data quickly. Commercially available software like school management systems offer modules for fee tracking, but they may be costly and overly complex for small institutions. Some institutions use basic database applications that lack robust security features and efficient reporting tools. These systems often fail to support partial payments, automatic dues calculation, or user-based access control. In comparison, the Student Fee Management System proposed in this project focuses on providing a secure, user-friendly, and cost-effective solution tailored to the specific needs of fee processing, payment tracking, and report generation. It aims to overcome limitations found in manual and existing digital methods by integrating structured data handling, role-based access, and automated calculations

1.5. Gap Analysis

Despite the availability of various fee management systems, several gaps exist that limit their effectiveness in real-world educational institutions. Many existing solutions are either too complex or expensive for small to medium-sized schools, making them difficult to adopt. Manual and semi-digital systems often suffer from data redundancy, errors in fee calculation, and delays in report generation. Security is another concern, as some systems lack proper user authentication and role-based access control, increasing the risk of unauthorized access. Additionally, most existing systems do not efficiently handle partial payments, late fee fines, or automatic tracking of defaulters. The proposed Student Fee Management System aims to bridge these gaps by providing a cost-effective, secure, and automated solution that ensures accurate fee tracking, timely report generation, and streamlined management of student financial records.

1.6. Proposed Solution

The proposed solution is a Student Fee Management System designed to automate and simplify fee-related operations in educational institutions. This system provides secure access through role-based authentication, allowing administrators and accounts officers to manage student records and fee transactions efficiently. It enables easy addition, updating, and searching of student information, as well as the management of classes, semesters, and academic sessions. The system allows for defining fee structures, recording full or partial payments, calculating dues and fines automatically, and generating detailed reports and receipts. Data is stored securely using file handling or database methods, with backup and recovery mechanisms to ensure integrity. Overall, this solution addresses the limitations of existing systems by offering accuracy, security, efficiency, and user-friendly operation for effective fee management.

1.7. Project Plan

Project Title: Fee Management System

1. Project Overview

The Student Fee Management System is designed to automate and manage student fee records in educational institutions. It provides secure access for different user roles, streamlines fee collection, calculates dues and fines, and generates reports and receipts. The system reduces manual work, prevents errors, and improves transparency in fee management.

2. Project Objectives

- Develop a secure, user-friendly fee management system.
- Automate the process of adding, updating, and managing student records.
- Record and process full or partial fee payments efficiently.
- Calculate dues and fines automatically.
- Generate reports, receipts, and financial summaries accurately.
- Ensure data safety through proper storage, backup, and recovery.

3. Project Scope

Included:

- User authentication and role management.
- Student, class, and semester management.
- Fee structure creation and modification.
- Fee payment processing (full and partial).
- Dues and fine calculation.
- Report and receipt generation.
- Data storage and backup.

Excluded:

- Online fee payment integration.
- Mobile or web-based access (current version is desktop-based).

4. Project Methodology

The project follows a **Waterfall Model** approach:

1. **Requirement Analysis** – Understand user needs and define system modules.
2. **System Design** – Design architecture, data flow, WBS, and diagrams.
3. **Implementation** – Develop modules using C++ and file handling.
4. **Testing** – Unit, integration, and system testing to ensure reliability.
5. **Deployment** – System installation and live data initialization.
6. **Documentation** – Prepare user manual, technical document, and final report.

5. Project Phases & Activities

Phase 1: Requirement Analysis

- Identify user roles (Admin, Accounts Officer).
- Gather functional and non-functional requirements.
- Define modules and sub-modules.

Phase 2: System Design

- Create Work Breakdown Structure (WBS).
- Design flowcharts and module diagrams.
- Plan data storage structure (files or database).

Phase 3: Implementation

- Develop User Authentication Module.
- Implement Student Management Module.
- Implement Class/Semester Management Module.
- Implement Fee Structure & Payment Modules.
- Implement Dues & Fine Management Module.
- Implement Reports & Receipts Module.

- Implement Data Storage & Backup Module.

Phase 4: Testing

- Unit testing of individual modules.
- Integration testing of all modules together.
- Identify and fix bugs.

Phase 5: Deployment

- Final system integration.
- Initialize student and fee data.
- Run system in live environment.

Phase 6: Documentation

- Prepare User Manual.
- Prepare Technical Documentation.
- Compile Project Report.

6. Project Timeline

Phase	Duration
Requirement Analysis	1 week
System Design	1 week
Implementation	3 weeks
Testing	1 week
Deployment	3 days
Documentation	1 week

7. Resources Required

- **Software:** C++ Compiler (Dev-C++ / CodeBlocks)
- **Hardware:** PC / Laptop
- **Human Resource:** Student / Developer
- **Storage:** File handling or local database

8. Risk Management

- **Data loss risk** – Handled with backup module.
- **Logical errors** – Reduced by unit and integration testing.
- **Time constraints** – Managed through phased timeline.

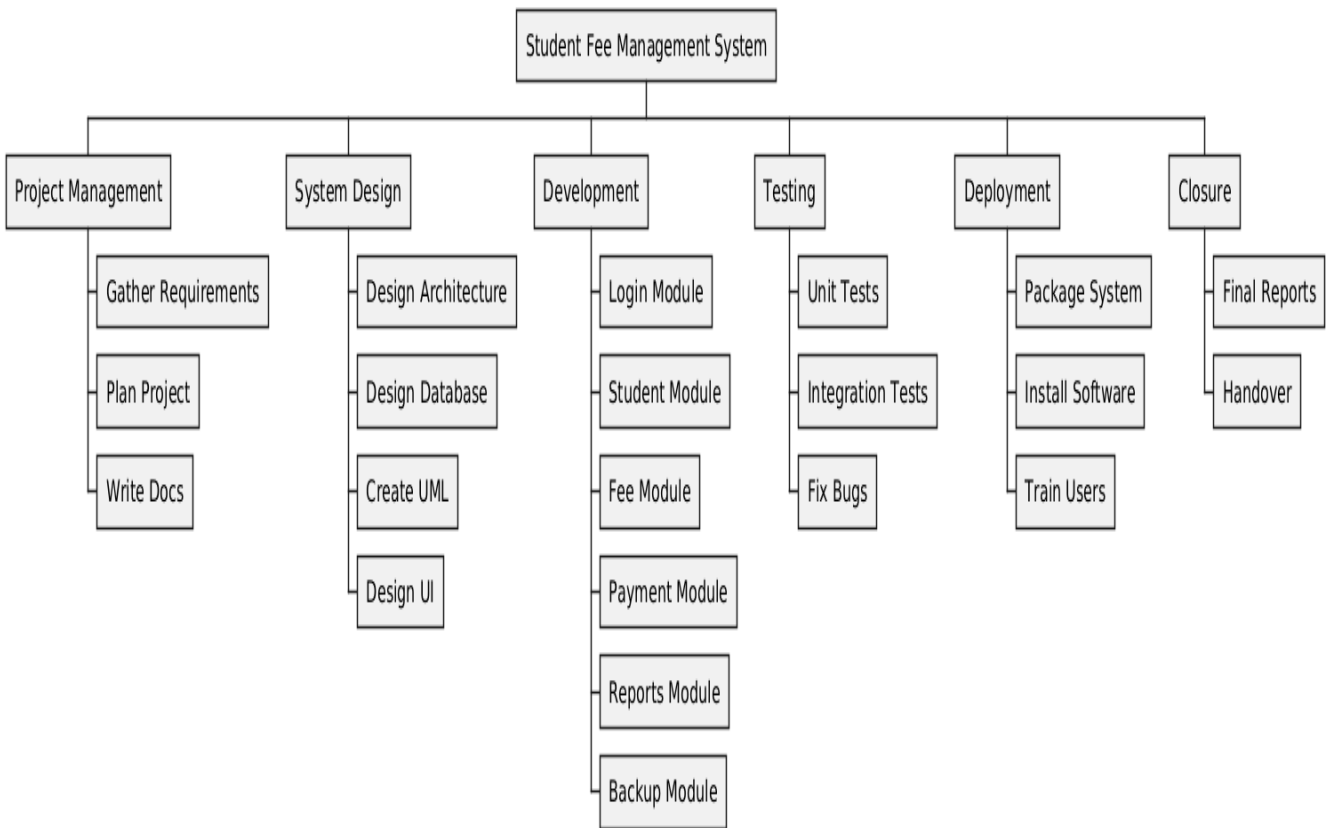
9. Deliverables

- Fully functional Fee Control System.
- Source code and executable.
- User and technical manuals.
- Project report with diagrams and screenshots.

10. Conclusion

The project plan provides a roadmap for the successful development of the Fee Control System. By following the structured phases, proper resource allocation, and testing procedures, the project aims to deliver a reliable, efficient, and secure solution for fee management in educational institutions.

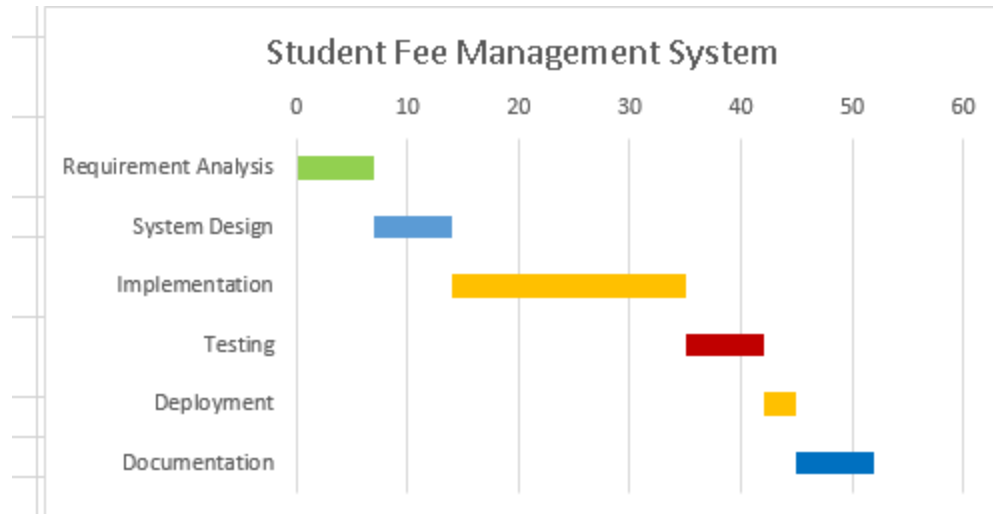
1.7.1. Work Breakdown Structure



1.7.2. Roles & Responsibility Matrix

Module / Task	Accountable	Tester
1. User Authentication & Role Management	Muhammad Furqan	Hafiz Abdul Wahab
2. Student Management	Muhammad Furqan	Hafiz Abdul Wahab
3. Class / Semester Management	Muhammad Furqan	Hafiz Abdul Wahab
4. Fee Structure Management	Muhammad Furqan	Hafiz Abdul Wahab
5. Fee Payment Processing	Muhammad Furqan	Hafiz Abdul Wahab
6. Dues & Fine Receipts	Muhammad Furqan	Hafiz Abdul Wahab
7. Data Storage & Backup	Muhammad Furqan	Hafiz Abdul Wahab
8. Testing & Deployment	Muhammad Furqan	Hafiz Abdul Wahab
9. Documentation	Muhammad Furqan	Hafiz Abdul Wahab

1.7.3. Gantt Chart



1.8. Report Outline

Chapter 1: Introduction

- 1.1 Background
- 1.2 Motivations and Challenges
- 1.3 Goals and Objectives
- 1.4 Proposed Solution
- 1.5 Report Outline

Chapter 2: SRS

- 2.1 Purpose and Scope
- 2.2 Product Functions
- 2.3 User Classes
 - Admin (full access)
 - Accounts Staff (limited access)
- 2.4 System Features
- 2.5 Nonfunctional Requirements (performance, security, usability)

Chapter 3: Use Case Analysis

- 3.1 Use Case Model

3.2 Use Case Descriptions

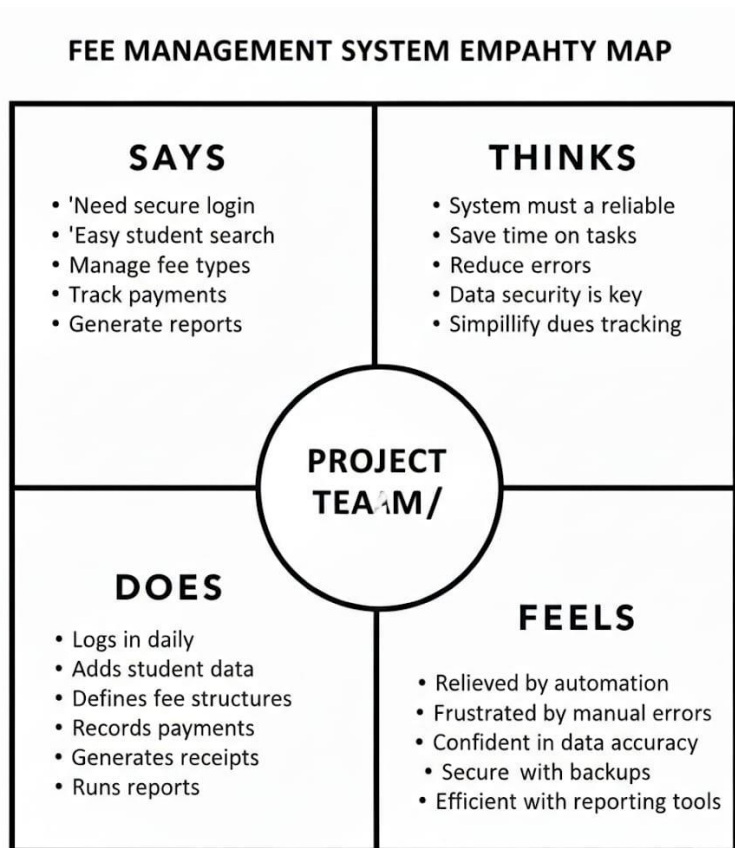
Chapter 4: System Design

- 4.1 Architecture Diagram
- 4.2 ERD with Data Dictionary
- 4.3 Class Diagram
- 4.4 Sequence Diagrams
- 4.5 Operation Contracts
- 4.6 Activity Diagram
- 4.7 State Transition Diagram
- 4.8 Component Diagram
- 4.9 DFD Level 0 and Level 1

Chapter 6: Testing

- 6.1 Use Case Testing
- 6.2 Equivalence Partitioning
- 6.3 Boundary Value Analysis
- 6.4 Unit Testing
- 6.5 Integration Testing
- 6.6 Performance Testing

1.9. Empathy Map



Chapter 2

Software Requirement Specifications

Chapter 2: Software Requirement Specifications

2.1. Introduction

2.1.1. Purpose

The purpose of this document is to specify the software requirements for the Student Management and Fee Processing System. This system is designed to automate student record management, class/semester handling, fee structure definition, fee payment processing, dues and fine calculation, and generation of reports and receipts. The software also ensures secure user access with roles for Admin and Accounts Officer/Staff.

Scope of the Product:

This SRS describes the entire Student Management and Fee Processing System, including the following modules:

1. **User Authentication & Roles** – Secure login for Admin and Accounts Officer/Staff with role-based access control and logout functionality.
2. **Student Management** – Add, update, view, and search student records.
3. **Class / Semester Management** – Manage classes/semesters, assign students, and define academic sessions.
4. **Fee Structure Management** – Define fee types (tuition, lab, library), set semester-wise fees, and update amounts.
5. **Fee Payment Processing** – Record full or partial payments, auto-generate payment dates, and maintain payment history.
6. **Dues & Fine Management** – Calculate remaining dues, late fees, and identify defaulters.
7. **Reports & Receipts** – Generate fee receipts, paid students report, defaulter report, and total collection report.
8. **Data Storage & Backup** – Handle file/database storage, perform backups, restore data, and ensure data integrity.

2.1.2. Document Conventions

This SRS document follows a structured numbering format (e.g., 1.0, 1.1, 1.1.1) for clear organization. Headings are written in bold to highlight important sections. Each requirement is uniquely identified and assigned a priority level (High, Medium, and Low). Higher-level requirement priorities are inherited by their detailed requirements unless stated otherwise. Consistent terminology is used throughout the document to avoid confusion.

2.1.3. Intended Audience and Reading Suggestions

This SRS document is intended for various readers involved in the system:

- **Developers:** To understand functional and non-functional requirements for implementation.
- **Project Managers:** To monitor progress and verify that requirements are met.
- **Testers:** To design test cases and ensure the system works as specified.
- **Users / Staff:** To understand system features and workflows.
- **Documentation Writers:** To create manuals and help guides.

The document is organized into modules, starting with an overview, system features, functional and non-functional requirements, and data management details.

Reading Suggestions:

- Begin with the overview sections to understand the purpose and scope.
- Developers and testers can then focus on functional requirements and module details.
- Project managers and users can refer to system features, reports, and user roles

2.1.4. Product Scope

The software is a **Student Fee Management System** designed to automate fee collection, payment tracking, and reporting for educational institutions. Its purpose is to reduce manual errors, save time, and improve accuracy in managing student records and fee transactions.

Benefits:

- Quick fee processing and receipt generation
- Easy identification of defaulters and dues
- Efficient report generation for management

The system supports the institution's goals of improving operational efficiency, maintaining accurate financial records, and providing better service to students and staff.

2.1.5. References

The following documents and sources were referred to in preparing this SRS:

1. **Vision and Scope Document** – Author: Muhammad Furqan, Version 1.0, February 2026, internal project document, available with project team.
2. **Use Case Document for Student Fee Management System** – Author: Muhammad Furqan, Version 1.0, February 2026, internal project document, available with project team.
3. **IEEE Standard for Software Requirements Specifications (IEEE 830-1998)** – Version 1.0, 1998, Source:

These references include internal documents prepared for this project and standard guidelines followed for theory-based SRS preparation.

2.2. Overall Description

2.2.1. Product Perspective

The Student Fee Management System is a new, self-contained software product designed to automate the management of student records, fee collection, payment tracking, and report generation for educational institutions. It is not a replacement for any existing system but can integrate with other administrative systems in the future.

Context and Interfaces:

- The system interacts with **users** (Admin, Accounts Officer) via a login interface.
- It stores data in an internal **database or file system**.
- It can generate **reports, receipts, and summaries** for management and staff.

Major Components of the System:

1. User Authentication & Roles
2. Student Management
3. Class/Semester Management
4. Fee Structure Management
5. Fee Payment Processing
6. Dues & Fine Management
7. Reports & Receipts
8. Data Storage & Backup

2.2.2. Product Functions

The Student Fee Management System will perform the following major functions:

- Authenticate users (Admin and Accounts Officer) and provide secure login and logout functionality.
- Manage student records including adding, updating, viewing, and searching students.
- Manage classes, semesters, and academic sessions.
- Define and update fee structures for different semesters and fee types.
- Record student fee payments (full or partial) and maintain payment history.
- Calculate remaining dues and apply late fee fines when applicable.
- Identify defaulters based on unpaid dues.
- Generate reports such as paid students report, defaulters report, total collection report, and fee receipts.
- Store data securely with backup and restore functionality.

These functions are organized into related modules such as authentication, student management, fee processing, reporting, and data management to ensure efficient and structured system operation.

2.2.3. User Classes and Characteristics

The Student Fee Management System has the following user classes:

- **Admin (High Priority):**
Has full access to the system. Manages users, fee structures, reports, and data backup. Requires moderate technical knowledge and uses the system regularly.
- **Accounts Officer / Staff (High Priority):**
Handles daily fee transactions, records payments, generates receipts, and checks dues. Has limited access compared to Admin and basic computer skills.
- **Management (Medium Priority):**
Views reports such as total collections and defaulters list. Has read-only access and uses the system occasionally.
- **Students (Low Priority – Indirect Users):**
Do not directly use the system but receive fee receipts and payment information.

These user classes are defined based on access level, responsibilities, and frequency of system usage.

2.2.4. Operating Environment

The Student Fee Management System will operate in a standard computer-based environment.

- **Hardware Platform:** Personal computer with minimum 4 GB RAM and standard storage capacity.
- **Operating System:** Windows 10 or later versions.
- **Software Requirements:** C++ compiler (such as Dev-C++ or Visual Studio) for program execution.
- **Database/File System:** Local file system or database for storing records.

The system is designed to run independently and coexist with other standard applications installed on the computer without conflict.

2.2.5. Design and Implementation Constraints

The development of the Student Fee Management System is subject to the following constraints:

- **Technology Constraint:** The system will be developed using C++ as required by the course project guidelines.
- **Hardware Limitation:** The system must operate on standard laboratory computers with limited memory and processing power.
- **Database Constraint:** Data will be stored using file handling instead of advanced database systems due to project scope limitations.
- **Security Constraint:** Role-based access control must be implemented to restrict unauthorized access.
- **Design Standards:** The system must follow structured programming principles and proper documentation standards as specified in the course.

These constraints limit the choice of tools, technologies, and design approaches used in the development of the system

2.2.6. User Documentation

The following user documentation components will be provided with the Student Fee Management System:

- **User Manual:** A document explaining system features, login process, student management, fee processing, and report generation.
- **Installation Guide:** Instructions for installing and running the software on a computer system.
- **Online Help / Help Section:** Basic help instructions within the system interface.
- **Tutorial Guide:** Step-by-step instructions for performing common tasks such as recording payments and generating reports.

The documentation will be delivered in PDF format and printed format if required, following standard academic documentation guidelines

2.2.7. Assumptions and Dependencies

The following assumptions and dependencies apply to the Student Fee Management System:

Assumptions:

- The system will be operated by users with basic computer knowledge.
- Required hardware and software (Windows OS and C++ compiler) will be available.
- Student and fee data provided to the system will be accurate.
- The system will be used in a single-institution environment.

Dependencies:

- The system depends on proper functioning of the operating system and hardware.

- It relies on file handling mechanisms for data storage.
- Future enhancements may depend on database integration or network availability.

If these assumptions are incorrect or change, system performance and functionality may be affected

2.3. External Interface Requirements

2.3.1. User Interfaces

The Student Fee Management System provides a console-based user interface for interaction between the user and the system.

- **Login Interface:**
Users (Admin and Accounts Officer) enter username and password to access the system.
- **Main Menu Interface:**
Displays available options such as Student Management, Fee Processing, Reports, and Logout.
- **Input Screens:**
Forms for adding students, recording payments, updating records, and managing fee structure.
- **Output Screens:**
Display student details, payment history, dues information, and generated reports

Interface Characteristics

- Menu-driven interface with numbered options.
- Clear prompts for user input.
- Error messages displayed for invalid input (e.g., wrong ID or incorrect password).
- Standard options like “Back to Main Menu” and “Exit” available on each screen.

The detailed user interface design will be documented separately in the User Interface Specification

2.3.2. Hardware Interfaces

The Student Fee Management System operates on standard computer hardware and does not require any specialized hardware components.

- **Supported Devices:**

Desktop or laptop computer with keyboard, monitor, and storage device (HDD/SSD).

- **Input Devices:**

Keyboard is used for entering student details, payment information, and system commands.

- **Output Devices:**

Monitor is used to display system menus, reports, receipts, and error messages.

A printer may optionally be used for printing fee receipts and reports.

- **Data Interaction:**

The system interacts with hardware through the operating system for data storage and retrieval from the local storage device.

No special communication protocols or external hardware interfaces are required, as the system runs locally on a standalone computer.

2.3.3. Software Interfaces

The Student Fee Management System interacts with the following software components:

- **Operating System:**

Windows 10 or later versions. The system relies on the OS for file handling, memory management, and execution support.

- **Programming Environment:**

Developed using C++ and compiled using Dev-C++ or Microsoft Visual Studio (C++ compiler).

- **File System / Data Storage:**

The system uses the local file system to store student records, fee details, and payment history. Data is read from and written to text or binary files. Data Exchange

Data Exchange

- **Input Data:**

- Student information (ID, name, semester)
- Fee structure details
- Payment records

- **Output Data:**

- Fee receipts
- Student records
- Dues and defaulter reports
- Total collection summaries

Nature of Communication

- The system communicates with software components through standard C++ file handling libraries and operating system services. No external APIs, network services, or third-party software components are used in the current version.
- No shared global data areas or special communication protocols are required, as the system operates as a standalone application.

2.3.4. Communications Interfaces

The Student Fee Management System is a standalone desktop application and does not require network or internet communication.

- No e-mail, web browser, or server communications are needed.
- No external message formatting or communication protocols (such as FTP or HTTP) are required.
- All data input and output are handled locally through the console interface and file storage.
- Security is limited to role-based access control within the application; no encryption or network security is needed.
- Since the system operates on a single machine, no synchronization or data transfer mechanisms are required.

2.4. System Features

The Student Fee Management System provides the following major system features:

1. User Authentication & Roles

Description: Secure access for different user types.

Key Functions:

- Admin login with full access.
- Accounts Officer / Staff login with limited access.
- Role-based permissions and secure logout.

2. Student Management

Description: Manage student records efficiently.

Key Functions:

- Add new student.

- Update student information.
- View student records.
- Search students by ID or name.

3. Class / Semester Management

Description: Organize students according to classes and semesters.

Key Functions:

- Create and manage classes/semesters.
- Assign students to specific semesters.
- Define academic sessions.

4. Fee Structure Management

Description: Define and manage fee types and amounts.

Key Functions:

- Define fee types (tuition, lab, library).
- Set semester-wise fees.
- Update fee amounts as needed.

5. Fee Payment Processing

Description: Record and track student payments.

Key Functions:

- Record full or partial payments.
- Auto-generate payment date.
- Maintain payment history.

6. Dues & Fine Management

Description: Track unpaid fees and penalties.

Key Functions:

- Calculate remaining dues.
- Apply late fee fines.
- Identify defaulters.

7. Reports & Receipts

Description: Generate all financial and student reports.

Key Functions:

- Fee receipt generation.
- Paid students report.
- Defaulters report.
- Total collection summary.

8. Data Storage & Backup

Description: Ensure data is stored safely and can be recovered.

Key Functions:

- File handling or database storage.
- Data backup and restore.
- Maintain data integrity.

2.4.1. System Feature 1

User Authentication & Roles

2.4.1.1. Description and Priority

This feature provides secure login and logout for system users, including Admin and Accounts Officer, with role-based access control. Users can only access functions allowed by their role, ensuring security and proper operation of the system.

Priority: High

Benefit: 9 – Essential for system security and controlled access.

Penalty: 8 – Unauthorized access could compromise data and system integrity.

Cost: 5 – Moderate effort to implement login validation and role management.

Risk: 6 – Security vulnerabilities may arise if not implemented correctly.

2.4.1.2. Stimulus/Response Sequences

Login Attempt

Stimulus: User enters username and password and clicks “Login.”

Response: System verifies credentials; grants access if correct or shows an error message if incorrect.

Select Feature

Stimulus: User chooses a feature from the main menu.

Response: System checks role and allows access if permitted; otherwise, displays “Access Denied.”

Logout

Stimulus: User selects “Logout.”

Response: System terminates session and returns to the login screen.

2.4.1.3. Functional Requirements

REQ-SF1-1: Allow Admin and Accounts Officer to log in using a valid username and password.

REQ-SF1-2: Verify credentials and grant access only if correct; show error otherwise.

REQ-SF1-3: Provide role-based access control.

REQ-SF1-4: Allow users to securely log out.

REQ-SF1-5: Display appropriate error messages for invalid inputs or unauthorized access.

2.4.2. System Feature 2

Student Management

2.4.2.1. Description and Priority

This feature allows users to manage student records by adding new students, updating information, viewing records, and searching students by ID or name.

Priority: High

Benefit: 9 – Essential for accurate student data management.

Penalty: 8 – Errors in student records affect all other system operations.

Cost: 5 – Moderate effort to implement forms, validation, and search.

Risk: 5 – Risk of data inconsistency if input validation is not applied.

2.4.2.2. Stimulus/Response Sequences

Add New Student

Stimulus: User selects “Add Student” and enters details.

Response: System validates input, saves the record, and confirms.

Update Student Information

Stimulus: User selects “Update Student” and enters student ID.

Response: System retrieves the record, allows edits, saves updated info, and confirms.

View Student Records

Stimulus: User selects “View Students.”

Response: System displays all student records.

Search Student

Stimulus: User selects “Search Student” and enters ID or Name.

Response: System displays matching student details or “Student not found.”

2.4.2.3. Functional Requirements

REQ-SF2-1: Allow adding new student records.

REQ-SF2-2: Allow updating existing student information.

REQ-SF2-3: Allow viewing all student records.

REQ-SF2-4: Allow searching students by ID or Name.

REQ-SF2-5: Validate input to prevent blank or invalid entries.

2.4.3. System Feature 3 (and so on)

Class / Semester Management

2.4.3.1. Description and Priority

This feature allows creating and managing classes/semesters, assigning students, and defining academic sessions.

Priority: High

Benefit: 9 – Essential for organizing students.

Penalty: 7 – Incorrect assignments affect fees and reports.

Cost: 5 – Moderate effort.

Risk: 5 – Risk of mismatched data if validation fails.

2.4.3.2. Stimulus/Response Sequences

Create Class/Semester

Stimulus: User enters class/semester details.

Response: System saves and confirms success.

Assign Students to Semester

Stimulus: User selects students for a semester.

Response: System links students and confirms.

Define Academic Sessions

Stimulus: User enters start/end dates.

Response: System saves session details.

View Classes/Semesters

Stimulus: User selects “View Classes/Semesters.”

Response: System displays all class/semester details.

2.4.3.3. Functional Requirements

REQ-SF3-1: Allow creating new classes and semesters.

REQ-SF3-2: Allow assigning students to specific semesters.

REQ-SF3-3: Allow defining academic sessions with dates.

REQ-SF3-4: Allow viewing all class, semester, and session details.

REQ-SF3-5: Validate all inputs for classes, semesters, and sessions.

2.4.4. System Feature 4

Fee Structure Management

2.4.4.1. Description and Priority

This feature allows defining fee types, setting semester-wise fees, and updating amounts.

Priority: High

Benefit: 9 – Essential for proper fee management.

Penalty: 8 – Incorrect fees disrupt payment and reports.

Cost: 5 – Moderate effort.

Risk: 5 – Risk of errors if validation fails.

2.4.4.2. Stimulus/Response Sequences

Define Fee Types

Stimulus: User enters fee type details.

Response: System saves and confirms.

Set Semester Fees

Stimulus: User enters fee amounts per semester.

Response: System saves and confirms.

Update Fee Amounts

Stimulus: User selects existing fee type and enters new amounts.

Response: System updates and confirms.

View Fee Structure

Stimulus: User selects “View Fee Structure.”

Response: System displays all fee types and amounts.

2.4.4.3. Functional Requirements

REQ-SF4-1: Allow defining fee types (tuition, lab, library).

REQ-SF4-2: Allow setting semester-wise fee amounts.

REQ-SF4-3: Allow updating fee amounts.

REQ-SF4-4: Allow viewing fee types and amounts.

REQ-SF4-5: Validate fee inputs (no blanks or negative values).

2.4.5. System Feature 5

Fee Payment Processing

2.4.5.1. Description and Priority

This feature records full or partial payments, auto-generates payment dates, and maintains history.

Priority: High

Benefit: 9 – Essential for tracking payments.

Penalty: 8 – Incorrect payments affect dues and reports.

Cost: 5 – Moderate effort.

Risk: 6 – Risk of errors if validation fails.

2.4.5.2. Stimulus/Response Sequences

Record Payment

Stimulus: User enters student ID, amount, and type.

Response: System validates, updates history, and confirms.

Auto-generate Payment Date

Stimulus: User records payment.

Response: System assigns current date automatically.

View Payment History

Stimulus: User selects “View Payment History.”

Response: System displays all payments and balances.

2.4.5.3. Functional Requirements

REQ-SF5-1: Record full or partial payments.

REQ-SF5-2: Auto-assign current date to payments.

REQ-SF5-3: Maintain payment history for each student.

REQ-SF5-4: Validate student ID and payment amount; show error if invalid.

2.4.6. System Feature 6

Dues & Fine Management

2.4.6.1. Description and Priority

This feature calculates remaining dues, applies late fees, and identifies defaulters.

Priority: High

Benefit: 9 – Essential for fee tracking and control.

Penalty: 8 – Errors affect financial records.

Cost: 5 – Moderate effort.

Risk: 5 – Risk if logic or dates are incorrect.

2.4.6.2. Stimulus/Response Sequences

Calculate Dues

Stimulus: User selects a student.

Response: System calculates remaining dues.

Apply Late Fee

Stimulus: System detects overdue payment.

Response: Late fees are added automatically.

Identify Defaulters

Stimulus: User requests defaulter list.

Response: System displays all students with unpaid dues.

2.4.6.3. Functional Requirements

REQ-SF6-1: Calculate remaining dues per student.

REQ-SF6-2: Apply late fees automatically.

REQ-SF6-3: List all defaulters.

REQ-SF6-4: Validate dates and payment amounts before fines.

2.4.7. System Feature 7

Reports & Receipts

2.4.7.1. Description and Priority

This feature generates fee receipts, paid students report, defaulter report, and total collection report.

Priority: High

Benefit: 9 – Essential for administration and record-keeping.

Penalty: 8 – Missing reports affect management decisions.

Cost: 5 – Moderate effort.

Risk: 5 – Risk of inaccurate reports if data is inconsistent.

2.4.7.2. Stimulus/Response Sequences

Generate Receipt

Stimulus: User selects student.

Response: System creates receipt with payment and student info.

Paid Students Report

Stimulus: User requests report.

Response: System displays fully paid students.

Defaulters Report

Stimulus: User requests report.

Response: System displays students with unpaid dues.

Total Collection Report

Stimulus: User requests report.

Response: System calculates total fees collected.

2.4.7.3. Functional Requirements

REQ-SF7-1: Generate individual fee receipts.

REQ-SF7-2: Provide paid students report.

REQ-SF7-3: Provide defaulters report.

REQ-SF7-4: Calculate total collection summary.

2.4.8. System Feature 8

Data Storage & Backup

2.4.8.1. Description and Priority

This feature ensures secure storage, backup, and restoration of all data.

Priority: High

Benefit: 9 – Essential for data integrity and reliability.

Penalty: 9 – Data loss disrupts system operations.

Cost: 5 – Moderate effort.

Risk: 5 – Risk if backup/restore is not maintained properly.

2.4.8.2. Stimulus/Response Sequences

Store Data

Stimulus: New or updated records.

Response: System saves and confirms success.

Backup Data

Stimulus: User selects backup.

Response: System copies all records to backup location.

Restore Data

Stimulus: User selects restore.

Response: System retrieves and restores data.

2.4.8.3. Functional Requirements

REQ-SF8-1: Store all student, fee, and payment data securely.

REQ-SF8-2: Backup all records to a safe location.

REQ-SF8-3: Restore data from backup when needed.

REQ-SF8-4: Validate data integrity during backup and restore

2.5. Other Nonfunctional Requirements

2.5.1. Performance Requirements

- The system shall respond to any user action (login, search, view, add/update records) within 2 seconds under normal load.
- The system shall handle at least 500 student records without noticeable performance degradation.
- Searching for a student by ID or Name shall return results within 1 second.
- Fee calculation, dues, and fine updates shall be completed within 2 seconds per student.

- Reports (Paid students, Defaulters, Total collection) shall generate within 5 seconds for up to 500 records.
- The system shall maintain data integrity and consistency during simultaneous updates or backups.
- The system shall automatically save any record changes to the database to prevent data loss.

2.5.2. Safety Requirements

- The system shall prevent unauthorized access to student and financial data by enforcing login and role-based access control.
- The system shall validate all input (IDs, names, fee amounts, and dates) to prevent incorrect or harmful data entry.
- The system shall ensure data backup and restore mechanisms to prevent permanent loss in case of hardware failure.
- The system shall protect against accidental deletion of records by requiring user confirmation before deleting any data.
- The system shall maintain transaction integrity for fee payments and dues calculations to avoid financial errors.
- The system shall comply with standard data protection policies to safeguard personal student information.

2.5.3. Security Requirements

- The system shall require username and password authentication for all users.
- Different users shall have role-based access control (Admin, Accounts Officer) to restrict access to sensitive features.
- All student and financial data shall be securely stored and protected from unauthorized access.

- The system shall log all login attempts, updates, and deletions for audit purposes.
- Passwords shall be stored securely using encryption or hashing.
- The system shall prevent unauthorized modifications to fee structures, payments, and student records.
- Compliance with basic data privacy policies shall be maintained to protect personal student information.

2.5.4. Software Quality Attributes

- **Reliability:** The system shall operate correctly without failure for at least 30 days of continuous use.
- **Usability:** Users shall be able to navigate and perform all tasks with minimal training; interfaces shall be clear and consistent.
- **Maintainability:** System updates and bug fixes shall be possible without major changes to core functionality; code should follow standard conventions.
- **Performance:** The system shall respond to user actions (search, add, update, generate reports) within 2–5 seconds.
- **Security:** Data shall be protected via authentication, role-based access, and encryption.
- **Portability:** The software shall run on Windows 10/11 or higher and compatible hardware without modification.
- **Testability:** All functions shall be designed to allow easy testing for correctness and performance.
- **Robustness:** The system shall handle invalid inputs gracefully and display error messages without crashing.
- **Flexibility/Adaptability:** New fee types, semesters, or sessions can be added without redesigning the system.

2.5.5. Business Rules

Only Admin users can create or delete user accounts.

Only Admin or Accounts Officer can add, update, or delete student records.

Only Accounts Officer can define fee types, set fee amounts, and process payments.

Students cannot access the system directly; only staff can perform actions on their behalf.

Fee payments must be recorded with a valid student ID; system shall reject invalid IDs.

Late fees are automatically applied after the due date for each semester.

Reports and receipts can only be generated by authorized users (Admin or Accounts Officer).

All operations must be logged for audit purposes (login, updates, deletions, payments).

2.6. Other Requirements

Database Requirements: The system shall store all data in a structured file-based database or lightweight DBMS (e.g., SQLite) with proper indexing for fast retrieval.

Data Integrity: The system shall ensure consistency and accuracy of all student, fee, and payment records.

Backup and Recovery: The system shall provide manual backup and restore functionality to prevent data loss.

Legal and Privacy Requirements: The system shall comply with basic student data privacy principles; no personal data shall be shared without authorization.

Reusability: Code shall be modular, allowing reuse of functions like input validation, search, and reporting for future projects.

Internationalization: The system shall support English language; future updates may allow other languages without redesign.

Portability: The system shall run on Windows 10/11 with minimum hardware requirements (4GB RAM, 500MB storage)

Scalability: The system shall handle up to 1000 student records without significant performance degradation

Chapter 3

Use Case Analysis

Chapter 3: System Analysis

The proposed system is a Student Fee Management System designed to automate and manage student fee-related operations in an educational institution. The system helps administrators and accounts officers efficiently manage student records, fee structures, payments, dues, fines, reports, and data backup processes.

The system replaces manual record-keeping methods with a computerized solution to improve accuracy, efficiency, and data security.

Purpose of the System

The main purpose of this system is to:

- Maintain student records in an organized manner.
- Manage fee structures semester-wise.
- Record and process student fee payments.
- Calculate dues and apply late fines automatically.
- Generate reports and receipts.
- Ensure secure data storage and backup.

The system aims to reduce paperwork, minimize human errors, and provide quick access to information.

Scope of the System

The system will include the following modules:

- User Authentication & Role Management
- Student Management
- Class/Semester Management
- Fee Structure Management

- Fee Payment Processing
- Dues & Fine Management
- Reports & Receipt Generation
- Data Backup & Restore

The system will allow only authorized users (Admin and Accounts Officer) to access and perform operations based on their roles.

System Users (Actors)

Admin

- Manages user accounts and roles.
- Manages students and semesters.
- Generates reports.
- Performs data backup and restore.

Accounts Officer

- Manages fee structures.
- Processes fee payments.
- Calculates dues and fines.
- Generates fee-related reports and receipts.

Functional Requirements

- Allow users to log in and log out securely.
- Allow Admin to manage users.
- Allow adding, updating, viewing, and searching students.
- Allow managing classes and semesters.
- Allow defining and updating fee structures.
- Allow recording full or partial fee payments.

- Automatically calculate dues and fines.
- Generate reports and receipts.
- Provide backup and restore functionality.

Non-Functional Requirements

- The system should be secure and accessible only to authorized users.
- The system should provide fast response time.
- The system should maintain data integrity and accuracy.
- The system should support data backup and recovery.
- The user interface should be simple and user-friendly.

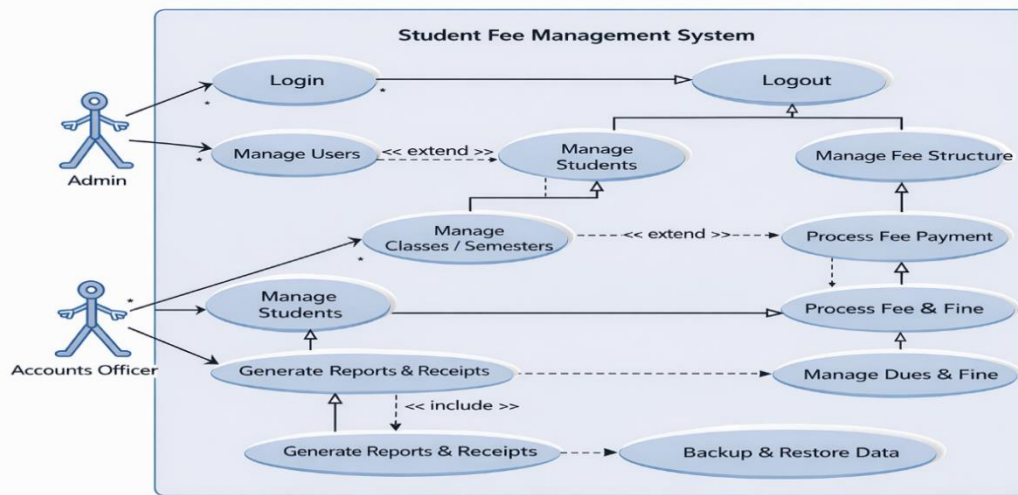
3.1. Use Case Model

A Use Case Model describes the functional behavior of the system. It shows how actors interact with the system to achieve their goals.

Components of a Use Case Model:

- **Actors:** Users or external systems interacting with the system (e.g., Admin, Accounts Officer).
- **Use Cases:** Functions or services provided by the system (e.g., Login, Manage Students, Process Fee Payment).
- **Relationships:** Associations between actors and use cases, as well as include, extend, and generalization relationships.

Use Case Diagram:



3.2. Use Case Descriptions

A Use Case Description provides a detailed explanation of a specific function or interaction in the system. It shows how an actor (user or external system) interacts with the system to achieve a particular goal. Each use case description typically includes: the use case name, unique ID, actors involved, preconditions, main flow, postconditions, and alternative flows. This ensures that the system's functional requirements are clearly documented and understood, and it helps developers, testers, and stakeholders know exactly how the system should behave in different scenarios.

Module 1: User Authentication & Roles

Use Case Name: Login

Use Case ID: UC-01

Use Case Description: Allows user to enter username and password to access the system.

Actor: Admin, Accounts Officer

Precondition: User account must already exist.

Postcondition: User is successfully logged in and redirected to dashboard.

Main Flow:

- User opens the login page.
- User enters username and password.
- System validates credentials.
- If credentials are valid, system grants access and redirects user to the dashboard.

Alternatives: If invalid credentials are entered, system displays error message and denies access.

Use Case Name: Logout

Use Case ID: UC-02

Use Case Description: Allows user to securely exit the system.

Actor: Admin, Accounts Officer

Precondition: User must be logged in.

Postcondition: User session ends and login page is displayed.

Main Flow:

- User clicks the logout button.
- System ends the user session.
- System redirects user to the login page.

Alternatives: If session times out, system automatically logs out user.

Use Case Name: Manage Users

Use Case ID: UC-03

Use Case Description: Admin creates, updates, or deletes system user accounts and assigns roles.

Actor: Admin

Precondition: Admin must be logged in.

Postcondition: User account information is stored or updated in system.

Main Flow:

- Admin opens the user management page.
- Admin selects to create, update, or delete a user account.
- Admin enters or updates user details.
- System validates input and saves changes.
- System confirms that the user account is created/updated/deleted successfully.

Alternatives: If duplicate username is entered, system shows validation error.

Module 2: Student Management

Use Case Name: Manage Students

Use Case ID: UC-04

Use Case Description: Add new students, update student information, view records, and search students.

Actor: Admin, Accounts Officer

Precondition: User must be logged in.

Postcondition: Student data is saved or updated successfully.

Main Flow:

- User opens the student management module.
- User selects Add, Update, View, or Search student.
- User enters or updates student details.
- System validates input and saves or updates student information.
- System displays a confirmation message.

Alternatives: If invalid student ID or missing fields are entered, system displays error message.

Module 3: Class / Semester Management

Use Case Name: Manage Classes/Semesters

Use Case ID: UC-05

Use Case Description: Define academic sessions, create semesters, and assign students.

Actor: Admin

Precondition: Admin must be logged in.

Postcondition: Semester and student assignment data saved in system.

Main Flow:

- Admin opens the class/semester management page.
- Admin selects to create a new semester or assign students.
- Admin enters semester details and selects students.
- System validates data and saves the semester and assignments.
- System confirms successful creation and assignment.

Alternatives: If semester already exists, system shows warning message.

Module 4: Fee Structure Management

Use Case Name: Manage Fee Structure

Use Case ID: UC-06

Use Case Description: Define fee types and set semester-wise fee amounts.

Actor: Accounts Officer

Precondition: Accounts Officer must be logged in.

Postcondition: Fee structure stored or updated successfully.

Main Flow:

- Accounts Officer opens the fee structure module.
- Officer selects to create or update fee types.
- Officer enters fee amounts for each semester.
- System validates data and stores the fee structure.
- System confirms that the fee structure is saved successfully.

Alternatives: If invalid fee amount is entered, system shows validation error.

Module 5: Fee Payment Processing

Use Case Name: Process Fee Payment

Use Case ID: UC-07

Use Case Description: Record full or partial payments and update student dues.

Actor: Accounts Officer

Precondition: Student record must exist in system.

Postcondition: Payment history updated and receipt generated.

Main Flow:

- Accounts Officer opens the fee payment module.
- Officer selects the student and enters payment details.
- System validates student record and payment amount.
- System updates payment history and student dues.
- System generates a receipt and displays confirmation.

Alternatives: If student ID not found, system displays error message.

Module 6: Dues & Fine Management

Use Case Name: Manage Dues & Fine

Use Case ID: UC-08

Use Case Description: Calculate remaining dues and apply late fine if applicable.

Actor: Accounts Officer

Precondition: Fee structure and payment records must exist.

Postcondition: Updated dues calculated and defaulter list generated.

Main Flow:

- Accounts Officer opens the dues and fine management module.
- System retrieves student fee and payment records.
- System calculates remaining dues and late fines (if any).

- System updates dues and generates defaulter list.
- System displays updated dues and defaulter list.

Alternatives: If no dues exist, system displays zero balance.

Module 7: Reports & Receipts

Use Case Name: Generate Reports & Receipts

Use Case ID: UC-09

Use Case Description: Generate fee receipts, paid student report, defaulter report, and total collection report.

Actor: Admin, Accounts Officer

Precondition: Payment data must be available.

Postcondition: Report displayed and ready for print or export.

Main Flow:

- User opens the reports module.
- User selects the type of report (fee receipt, paid student, defaulter, total collection).
- System retrieves relevant data from the database.
- System generates the report in the requested format.
- System displays report for viewing, printing, or export.

Alternatives: If no data available, system displays “No Record Found.”

Module 8: Data Storage & Backup

Use Case Name: Backup Data

Use Case ID: UC-10

Use Case Description: Create backup copy of system data.

Actor: Admin

Precondition: Admin must be logged in.

Postcondition: Backup file successfully created.

Main Flow:

- Admin opens the backup module.
- Admin selects to create a backup.
- System collects all system data and creates a backup file.
- System confirms that backup was created successfully.

Alternatives: If storage device is unavailable, system displays error message.

Use Case Name: Restore Data

Use Case ID: UC-11

Use Case Description: Restore system data from backup file.

Actor: Admin

Precondition: Valid backup file must exist.

Postcondition: System data restored successfully.

Main Flow:

- Admin opens the restore module.
- Admin selects a valid backup file.
- System validates backup file integrity.
- System restores all system data from the backup.
- System confirms successful restoration.

Alternatives: If backup file is corrupted, system shows restoration failure message.

Chapter 4

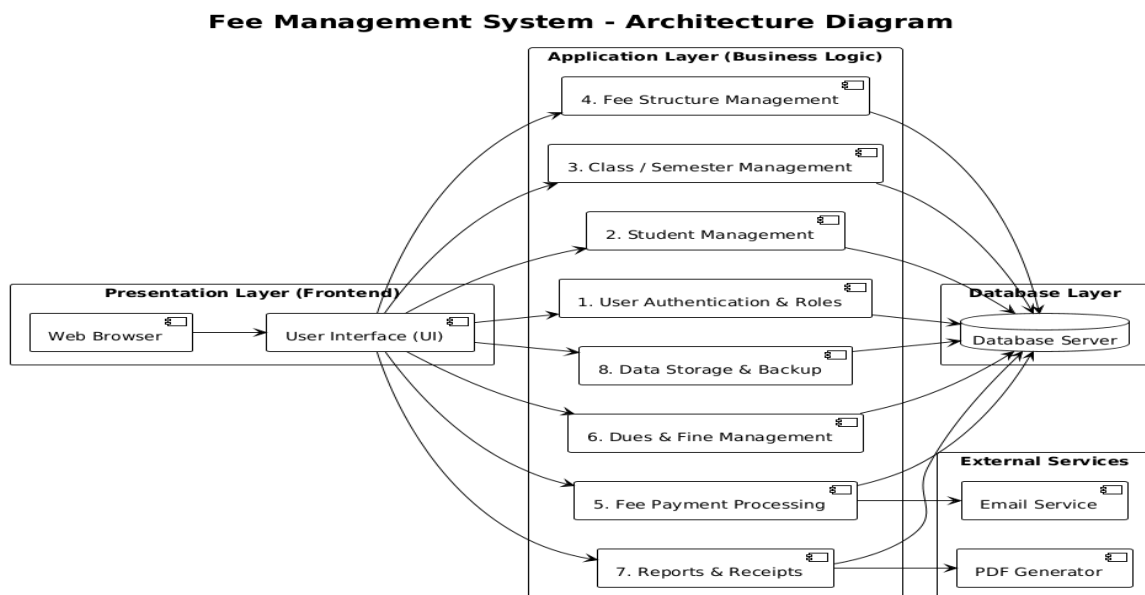
System Design

Chapter 4: System Design

This chapter presents the complete system design of the Fee Management System, including architectural design, UML diagrams, database design, and user interface design. It provides a detailed blueprint for implementing the system based on the requirements gathered and analyzed in the previous chapter. The design specifications described here serve as a guide for developers during the coding and implementation phase. This chapter covers class diagrams, sequence diagrams, activity diagrams, entity-relationship diagrams, and screen layouts. Each design component is explained with appropriate diagrams and descriptions to ensure a clear understanding of the system's structure and behavior.

4.1. Architecture Diagram

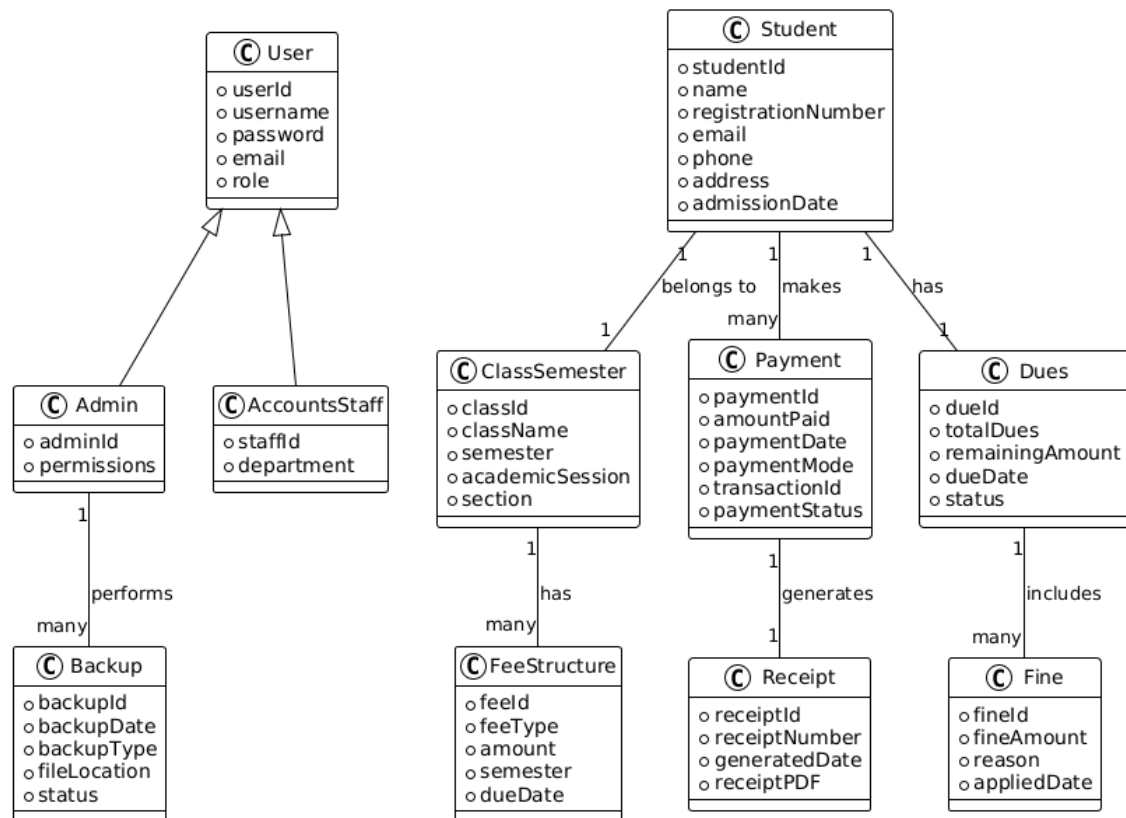
An architecture diagram is a high-level visual representation of a system's structure, showing the major components, their interactions, data flow, and the relationships between hardware and software layers. It provides an overview of how the system is organized, including presentation, business logic, and data storage layers, along with external interfaces and communication protocols.



4.2. Domain Model

A domain model is a conceptual visual representation of the key concepts, entities, attributes, and relationships within a specific problem domain. It illustrates the real-world objects and their associations, without including any implementation details such as methods or database-specific constraints. The domain model serves as a bridge between business requirements and system design.

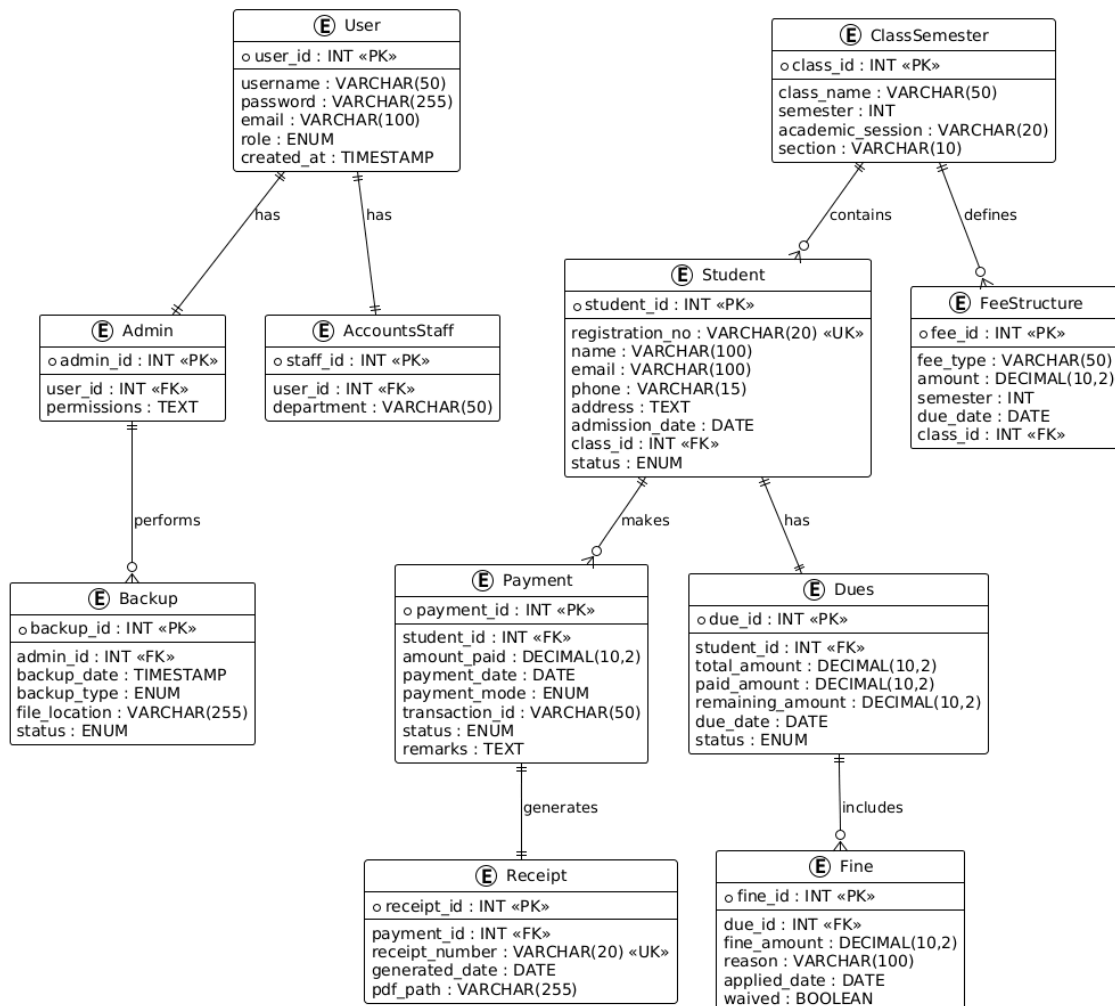
Fee Management System - Domain Model



4.3. Entity Relationship Diagram with data dictionary

An Entity Relationship Diagram (ERD) is a visual representation of the database structure that shows entities (tables), their attributes (fields), and the relationships between them. It illustrates how data is organized, including primary keys, foreign keys, cardinalities, and constraints. The ERD serves as a blueprint for database design and implementation. A data dictionary complements the ERD by providing detailed descriptions of each table, attribute, data type, constraints, and purpose within the system.

Fee Management System - Entity Relationship Diagram (ERD)



Data Dictionary*Table 1: User*

Attribute	Data Type	Constraint	Description
user_id	INT	PRIMARY KEY	Unique identifier for each user
username	VARCHAR(50)	NOT NULL	Login username
password	VARCHAR(255)	NOT NULL	Encrypted password
email	VARCHAR(100)	NOT NULL	User's email address
role	ENUM	NOT NULL	Role type (Admin, AccountsStaff)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation date

Table 2: Student

Attribute	Data Type	Constraint	Description
student_id	INT	PRIMARY KEY	Unique student identifier
registration_no	VARCHAR(20)	UNIQUE, NOT NULL	Unique registration number
name	VARCHAR(100)	NOT NULL	Student's full name
email	VARCHAR(100)	-	Student's email address
phone	VARCHAR(15)	-	Contact phone number
address	TEXT	-	Residential address
admission_date	DATE	NOT NULL	Date of admission
class_id	INT	FOREIGN KEY	References ClassSemester
status	ENUM	NOT NULL	Active/Inactive/Graduated

Table 3: ClassSemester

Attribute	Data Type	Constraint	Description
class_id	INT	PRIMARY KEY	Unique class identifier
class_name	VARCHAR(50)	NOT NULL	Name of class (e.g., BSCS, BBA)
semester	INT	NOT NULL	Semester number (1-8)

Attribute	Data Type	Constraint	Description
academic_session	VARCHAR(20)	NOT NULL	Academic year (e.g., 2024-2025)
section	VARCHAR(10)	-	Section (A, B, C)

Table 4: FeeStructure

Attribute	Data Type	Constraint	Description
fee_id	INT	PRIMARY KEY	Unique fee identifier
fee_type	VARCHAR(50)	NOT NULL	Type of fee (Tuition, Lab, Library)
amount	DECIMAL(10,2)	NOT NULL	Fee amount
semester	INT	NOT NULL	Applicable semester
due_date	DATE	NOT NULL	Payment due date
class_id	INT	FOREIGN KEY	References ClassSemester

Table 5: Payment

Attribute	Data Type	Constraint	Description
payment_id	INT	PRIMARY KEY	Unique payment identifier
student_id	INT	FOREIGN KEY	References Student
amount_paid	DECIMAL(10,2)	NOT NULL	Amount paid
payment_date	DATE	NOT NULL	Date of payment
payment_mode	ENUM	NOT NULL	Cash/Card/Bank Transfer
transaction_id	VARCHAR(50)	-	Reference transaction ID
status	ENUM	NOT NULL	Success/Pending/Failed
remarks	TEXT	-	Additional notes

Table 6: Dues

Attribute	Data Type	Constraint	Description
due_id	INT	PRIMARY KEY	Unique due identifier
student_id	INT	FOREIGN KEY	References Student

Attribute	Data Type	Constraint	Description
total_amount	DECIMAL(10,2)	NOT NULL	Total fee amount
paid_amount	DECIMAL(10,2)	DEFAULT 0	Amount already paid
remaining_amount	DECIMAL(10,2)	NOT NULL	Outstanding amount
due_date	DATE	NOT NULL	Last payment date
status	ENUM	NOT NULL	Paid/Partial/Unpaid

Table 7: Fine

Attribute	Data Type	Constraint	Description
fine_id	INT	PRIMARY KEY	Unique fine identifier
due_id	INT	FOREIGN KEY	References Dues
fine_amount	DECIMAL(10,2)	NOT NULL	Late fee amount
reason	VARCHAR(100)	-	Reason for fine
applied_date	DATE	NOT NULL	Date fine was applied
waived	BOOLEAN	DEFAULT FALSE	Fine waived or not

Table 8: Receipt

Attribute	Data Type	Constraint	Description
receipt_id	INT	PRIMARY KEY	Unique receipt identifier
payment_id	INT	FOREIGN KEY	References Payment
receipt_number	VARCHAR(20)	UNIQUE, NOT NULL	Auto-generated receipt number
generated_date	DATE	NOT NULL	Receipt generation date
pdf_path	VARCHAR(255)	-	File path for PDF receipt

Table 9: Backup

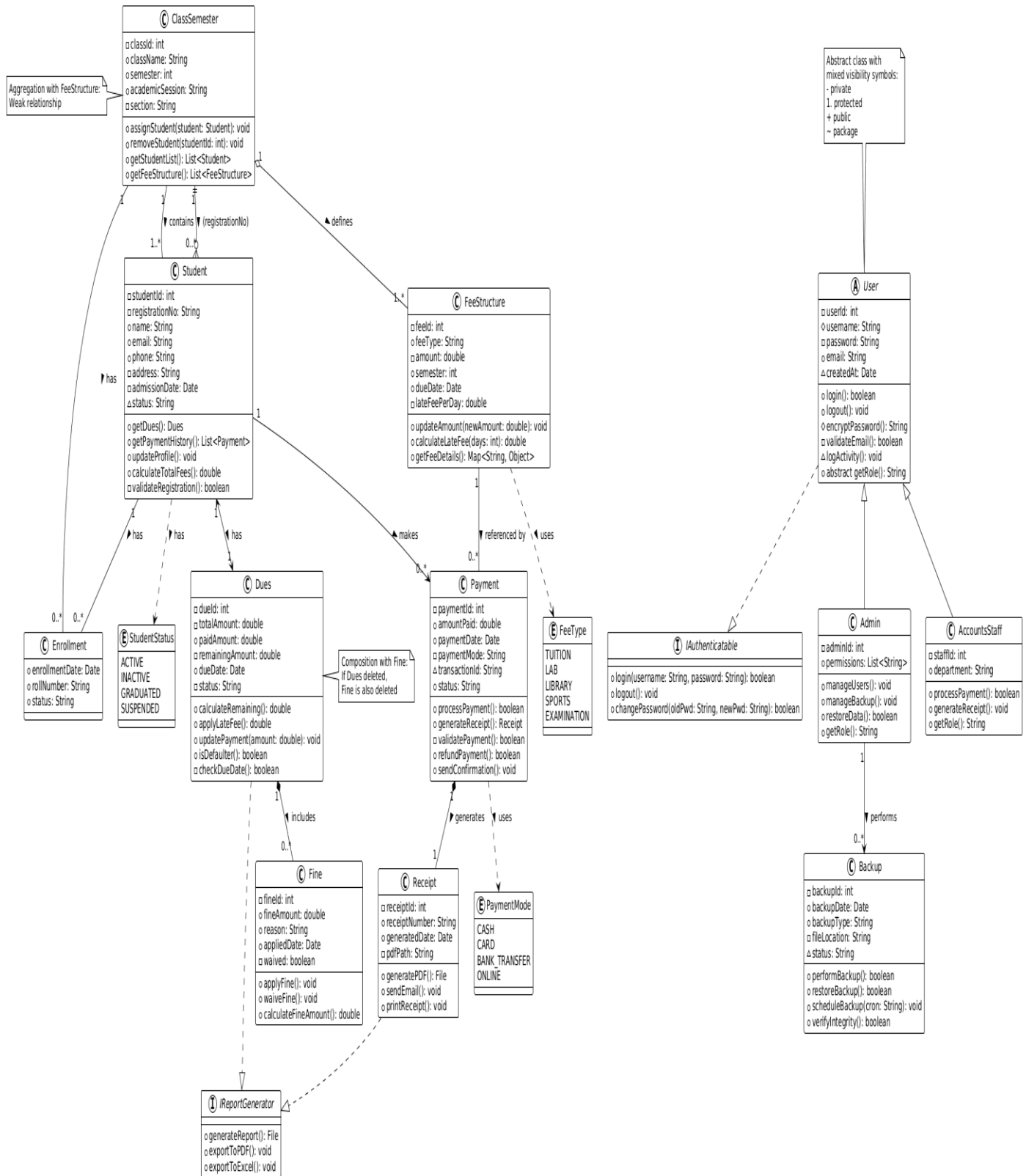
Attribute	Data Type	Constraint	Description
backup_id	INT	PRIMARY KEY	Unique backup identifier
admin_id	INT	FOREIGN KEY	References Admin

Attribute	Data Type	Constraint	Description
backup_date	TIMESTAMP	NOT NULL	Date and time of backup
backup_type	ENUM	NOT NULL	Full/Incremental/Auto
file_location	VARCHAR(255)	NOT NULL	Storage path of backup file
status	ENUM	NOT NULL	Success/Failed/InProgress

4.4. Class Diagram

A class diagram is a Unified Modeling Language (UML) structural diagram that shows the classes, attributes, methods, and relationships (such as inheritance, association, and dependency) within a system. It provides a static view of the system's architecture and serves as a blueprint for implementation

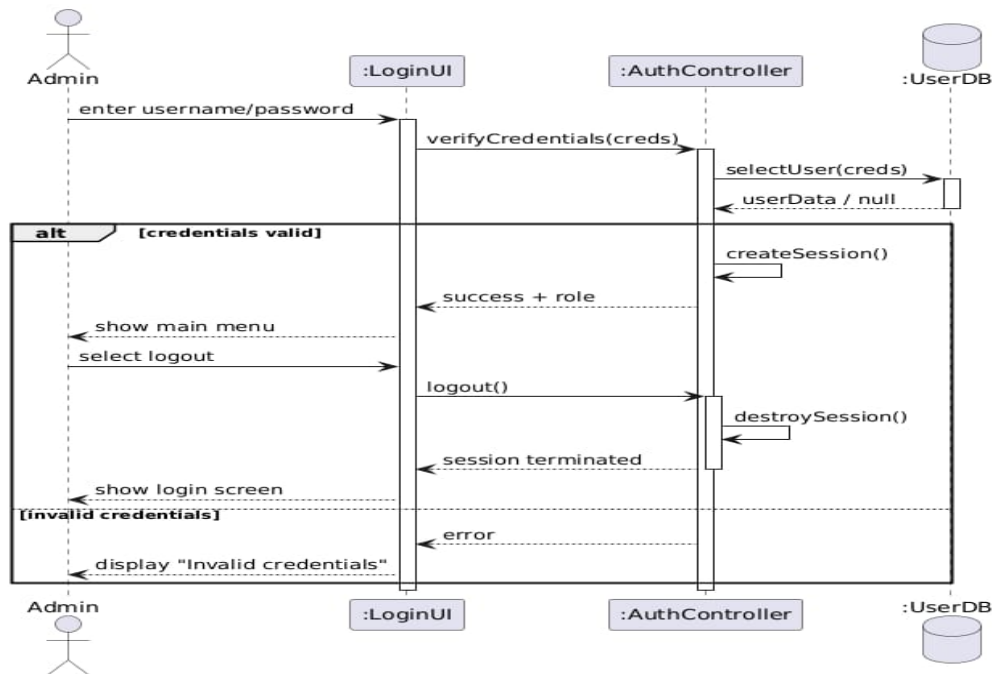
Fee Management System - Complete Class Diagram (All UML Symbols)



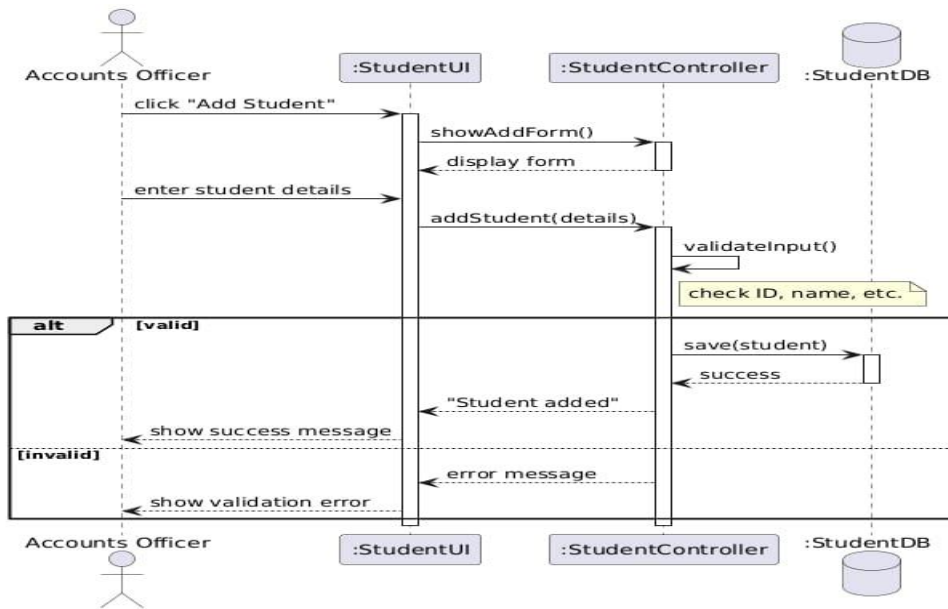
4.5. Sequence / Collaboration Diagram

A sequence diagram is a Unified Modeling Language (UML) interaction diagram that shows how objects interact with each other in a time-ordered sequence. It illustrates the flow of messages between objects, the order of operations, and the lifelines of participants during a specific use case or scenario.

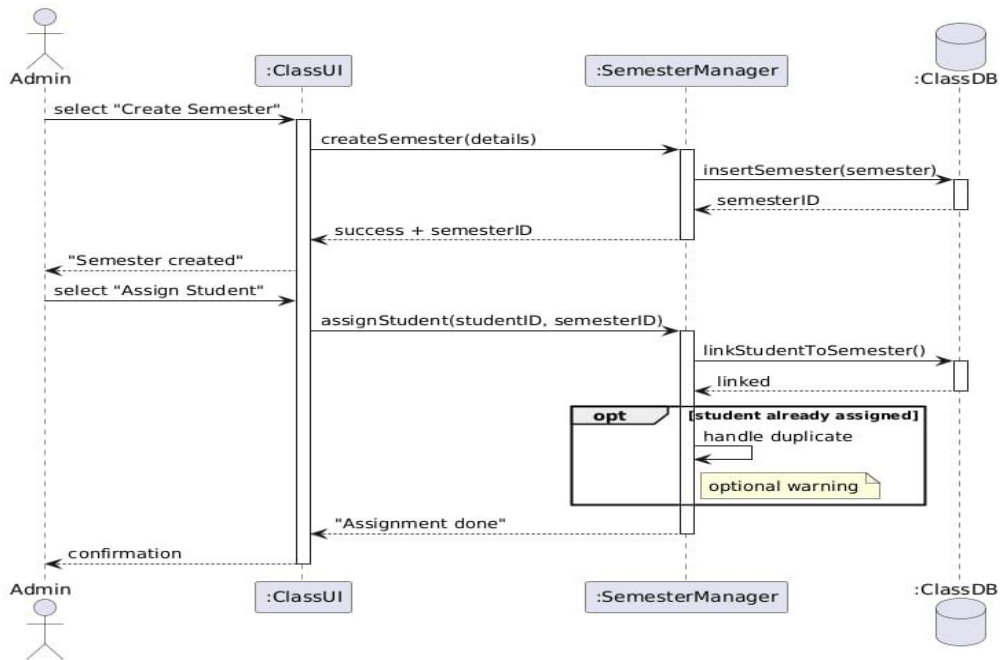
1. User Authentication & Roles (Login & Logout)



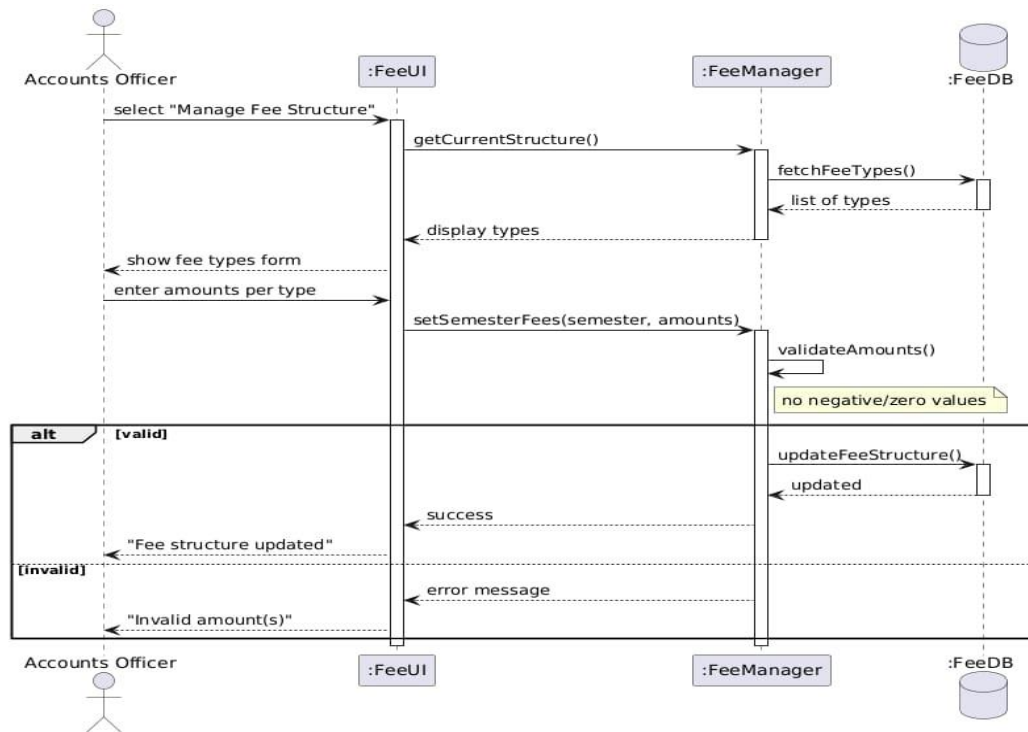
2. Student Management (Add Student)



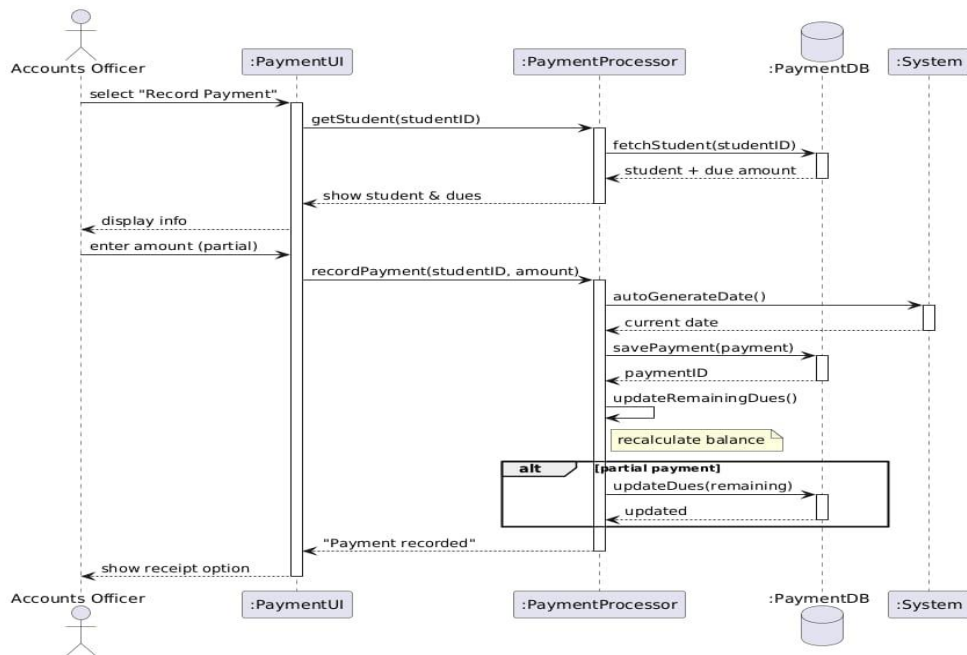
3. Class / Semester Management (Create Semester & Assign Student)



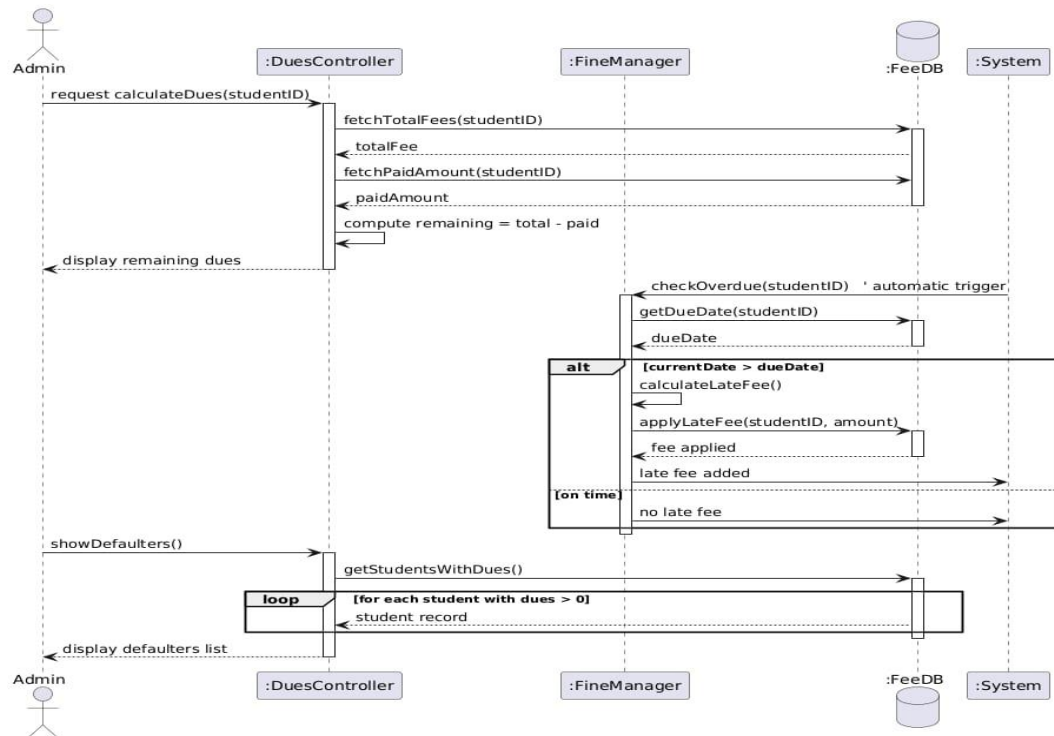
4. Fee Structure Management (Set Semester Fees)



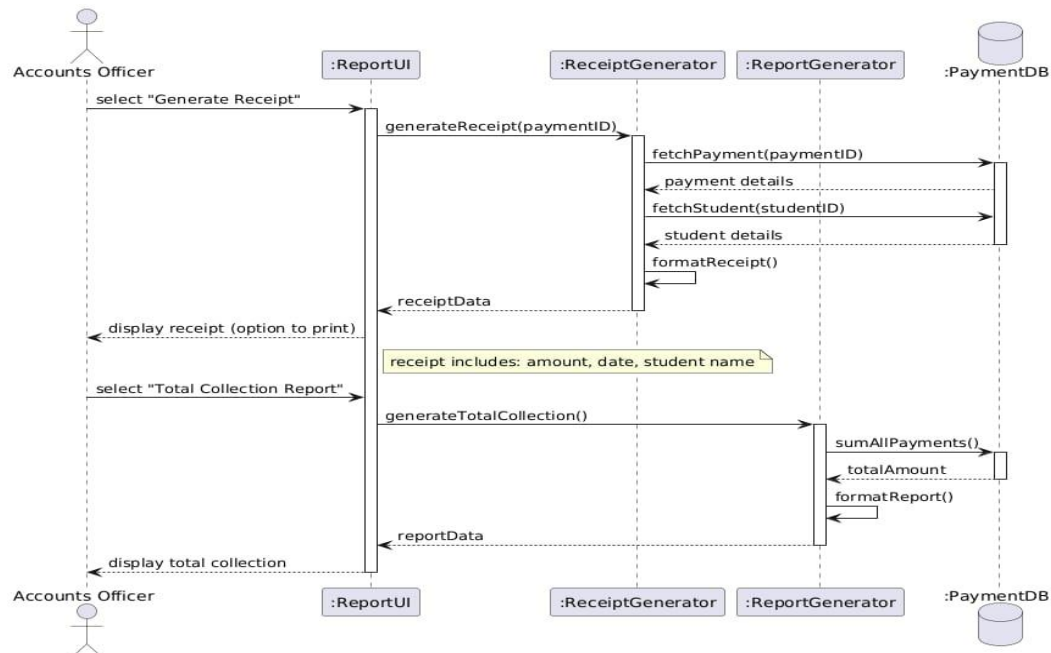
5. Fee Payment Processing (Record Partial Payment)



6. Dues & Fine Management (Calculate Dues, Apply Late Fee, List Defaulters)

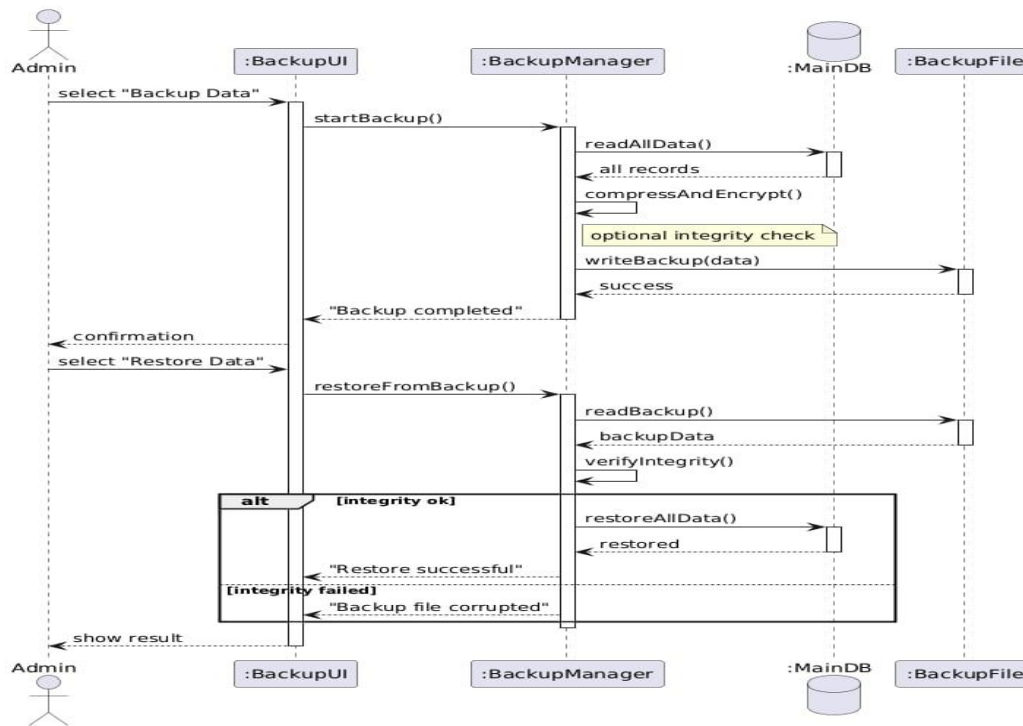


7. Reports & Receipts (Generate Receipt and Total Collection Report)



8. Data

8.Storage & Backup (Backup and Restore)



4.6. Operation contracts

An operation contract is a UML document that specifies the detailed behavior of a system operation by defining pre-conditions and post-conditions. It describes what happens when an operation is executed, without specifying how the operation is implemented. Operation contracts are derived from system sequence diagrams and help bridge the gap between high-level requirements and detailed design. Each contract includes an operation name, cross-references to use cases, pre-conditions that must be true before execution, and post-conditions that describe the state changes after execution.

Contract 1: Login

Element	Description
Operation:	login(username: String, password: String): boolean
Cross References:	Use Case: User Authentication

Element	Description
Pre-conditions:	System is running. User is not logged in. - A User object is validated against credentials. - If valid, a session is created.
Post-conditions:	- User's last login timestamp is updated. - Role-based permissions are loaded. - Login success flag is returned.

Contract 2: Add Student

Element	Description
Operation:	addStudent(studentDetails: StudentDTO): boolean
Cross References:	Use Case: Student Management
Pre-conditions:	Admin is logged in. Student with same registration number does not exist. - A new Student instance is created. - Student attributes are set (name, email, phone, address).
Post-conditions:	- Student is assigned a unique registration number. - Student is saved to the database. - A confirmation message is generated.

Contract 3: Assign Student to Class

Element	Description
Operation:	assignStudentToClass(studentId: int, classId: int): boolean
Cross References:	Use Case: Class / Semester Management
Pre-conditions:	Student exists. Class exists. Student is not already assigned to this class. - An association is created between Student and ClassSemester.
Post-conditions:	- Enrollment date is recorded. - Student's class list is updated.

Element	Description
---------	-------------

- Class' student count is incremented.
- Roll number is generated (if applicable).

Contract 4: Define Fee Structure

Element	Description
---------	-------------

Operation: defineFeeStructure(feeDetails: FeeStructureDTO): boolean

Cross References: Use Case: Fee Structure Management

Pre-conditions: Admin is logged in. Class exists. Fee type is valid.

- A new FeeStructure instance is created.
- Fee type, amount, semester, and due date are set.

Post-conditions:

- Fee structure is linked to the specified class.
- Fee structure is saved to the database.
- Duplicate fee structures are prevented.

Contract 5: Process Payment

Element	Description
---------	-------------

Operation: processPayment(paymentDetails: PaymentDTO): Receipt

Cross References: Use Case: Fee Payment Processing

Pre-conditions: Student exists. Amount paid > 0. Payment mode is valid.

- A new Payment instance is created.
- Payment amount, date, mode, and transaction ID are recorded.
- Student's due record is updated (paid_amount increases,

Post-conditions: remaining_amount decreases).

- If remaining_amount == 0, due status becomes "Paid".
- A Receipt is generated with a unique receipt number.
- Payment history is updated.

Contract 6: Calculate Dues

Element	Description
Operation:	calculateDues(studentId: int): Dues
Cross References:	Use Case: Dues & Fine Management
Pre-conditions:	Student exists. Fee structure for student's class exists. - Total fee amount is calculated from FeeStructure. - Previous payments are retrieved.
Post-conditions:	- Paid amount is summed from Payment history. - Remaining amount = total_amount - paid_amount. - A Dues object is created/updated with calculated values. - Due date is set based on fee structure.

Contract 7: Apply Late Fine

Element	Description
Operation:	applyLateFine(duelId: int): double
Cross References:	Use Case: Dues & Fine Management
Pre-conditions:	Dues exist. Due date has passed. Remaining amount > 0. - Number of late days is calculated. - Fine amount = days × late_fee_per_day.
Post-conditions:	- A new Fine instance is created and linked to Dues. - Fine amount is added to remaining dues. - Student is marked as defaulter if applicable. - Fine history is updated.

Contract 8: Generate Fee Receipt

Element	Description
Operation:	generateReceipt(paymentId: int): Receipt

Element	Description
Cross	Use Case: Reports & Receipts
References:	
Pre-conditions:	Payment exists. Payment status is "Success".
	- A unique receipt number is generated.
	- Receipt includes: student name, registration number, payment amount,
Post-	date, payment mode.
conditions:	- Receipt is saved with generated date.
	- PDF file is created and stored.
	- Receipt object is returned.

Contract 9: Generate Defaulter Report

Element	Description
Operation:	generateDefaulterReport(): List<Student>
Cross References:	Use Case: Reports & Receipts
Pre-conditions:	Admin/AccountsStaff is logged in.
	- All students with remaining_amount > 0 are retrieved.
	- Students with due_date passed are filtered.
Post-conditions:	- Students are sorted by overdue days.
	- Report includes student name, class, due amount, overdue days.
	- Report can be exported to PDF/Excel.

Contract 10: Perform Database Backup

Element	Description
Operation:	performBackup(backupType: String): boolean
Cross References:	Use Case: Data Storage & Backup
Pre-conditions:	Admin is logged in. Sufficient storage space is available.

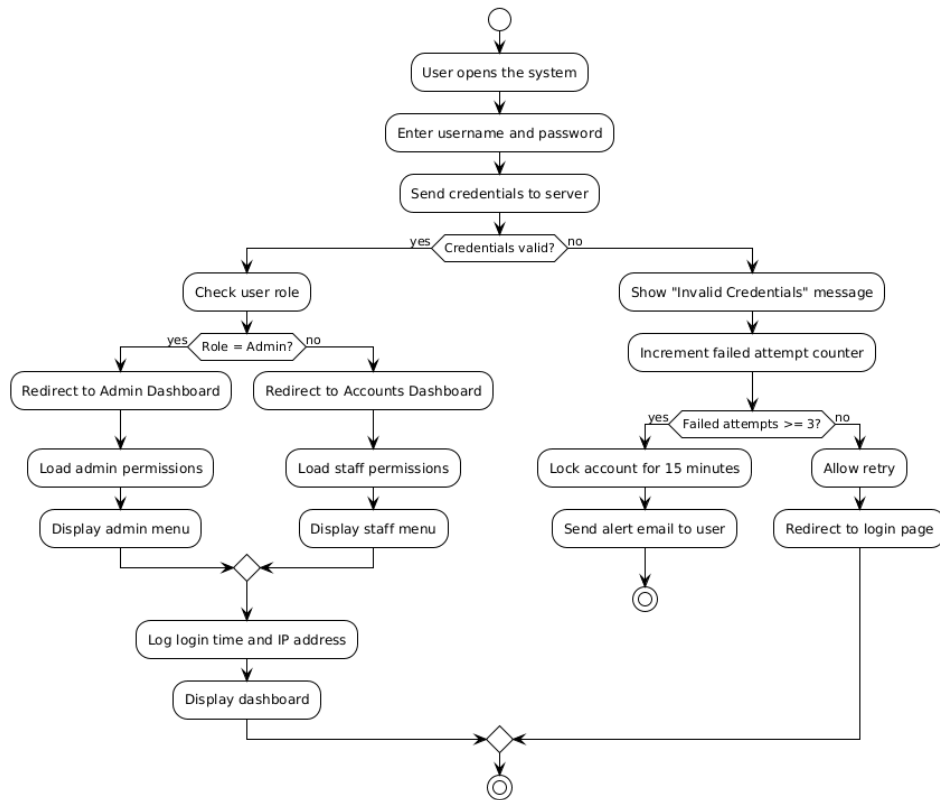
Element	Description
Post-conditions:	- Database is locked for consistency (if needed).
	- Backup file is created with timestamp.
	- Backup file is stored at specified location.
	- A Backup record is created with status "Success".
	- Old backups are rotated (if configured).
	- Backup completion notification is logged.

4.7. Activity Diagram

An activity diagram is a UML behavioral diagram that shows the flow of control from one activity to another within a system. It represents the sequential steps, decision points, parallel processing, and synchronization of activities. Activity diagrams are used to model workflows, business processes, and the logic of use cases. They help visualize the dynamic behavior of a system, including branching (using decision nodes), concurrency (using fork and join nodes), and loops.

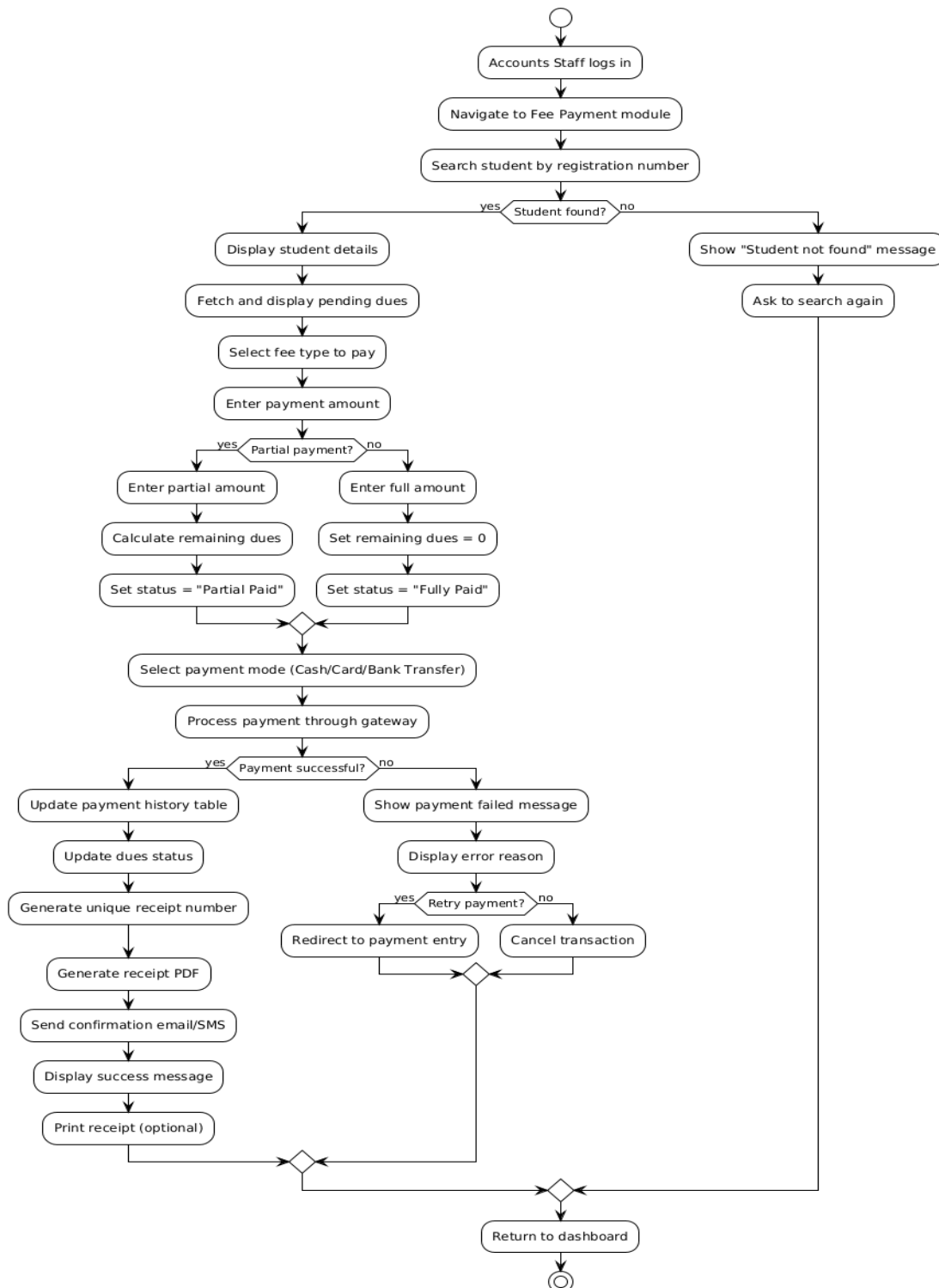
Activity Diagram 1: User Authentication

Fee Management System - Activity Diagram (User Authentication)



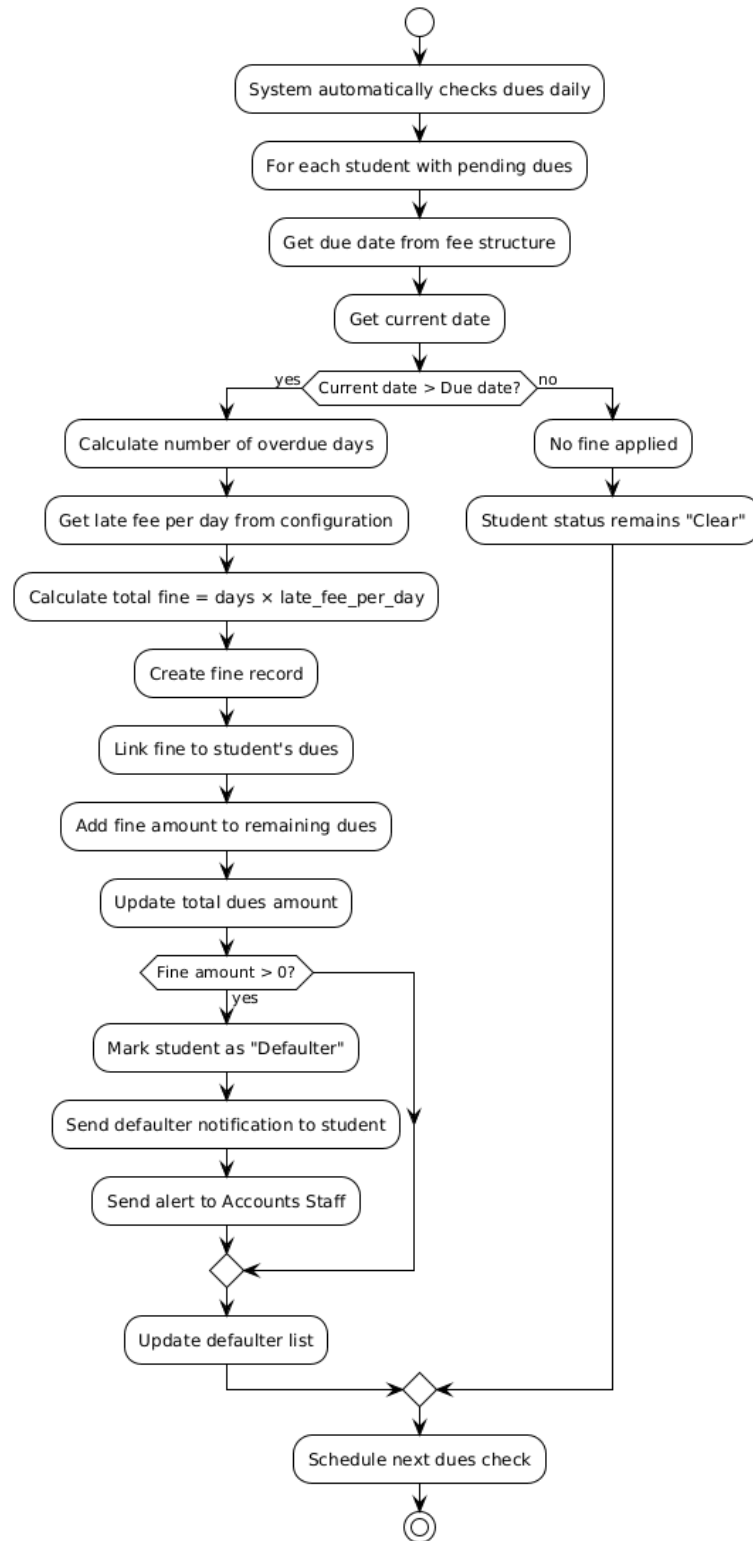
Activity Diagram 2: Fee Payment Processing

Fee Management System - Activity Diagram (Fee Payment Processing)

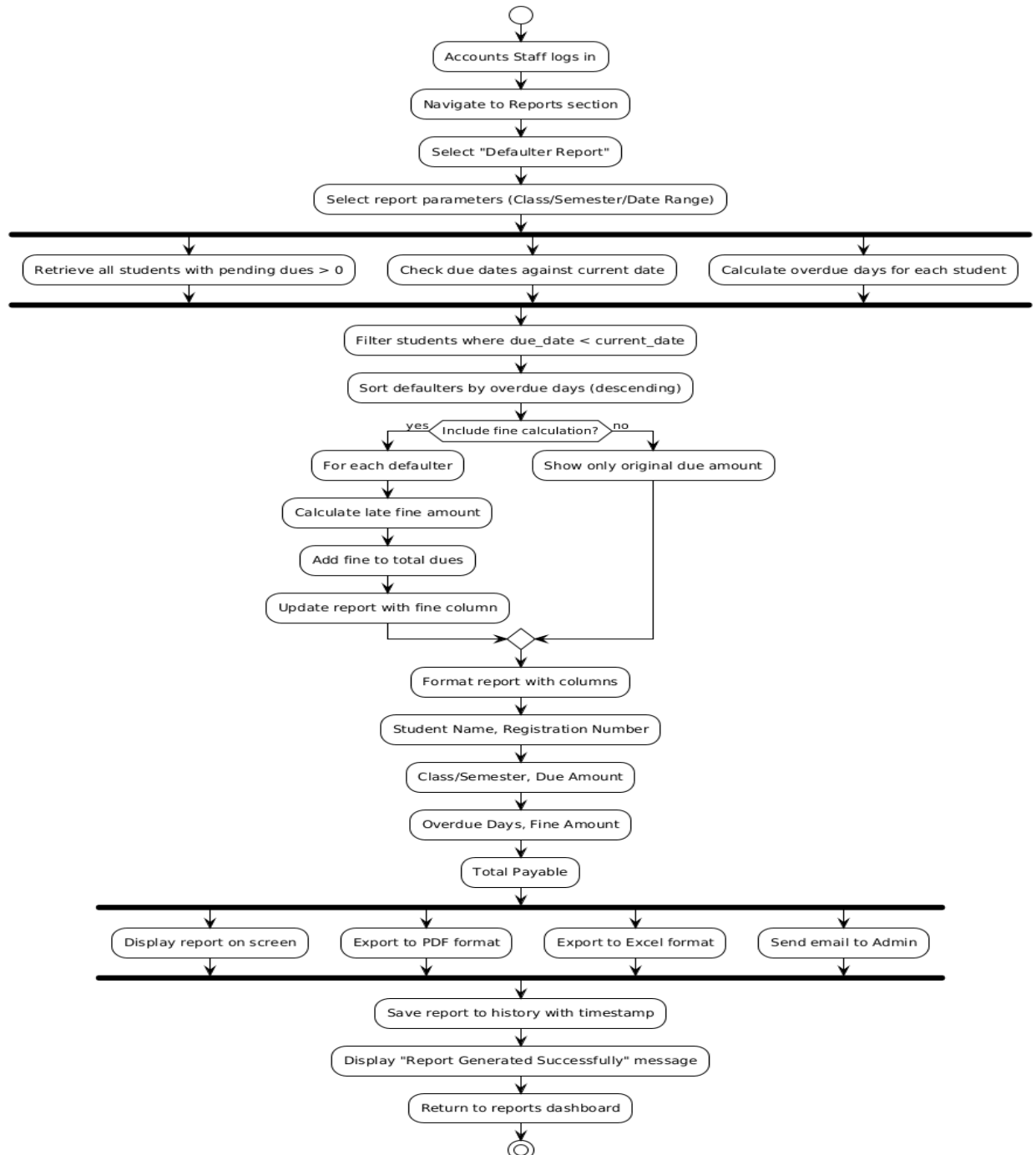


Activity Diagram 3: Dues & Fine Management

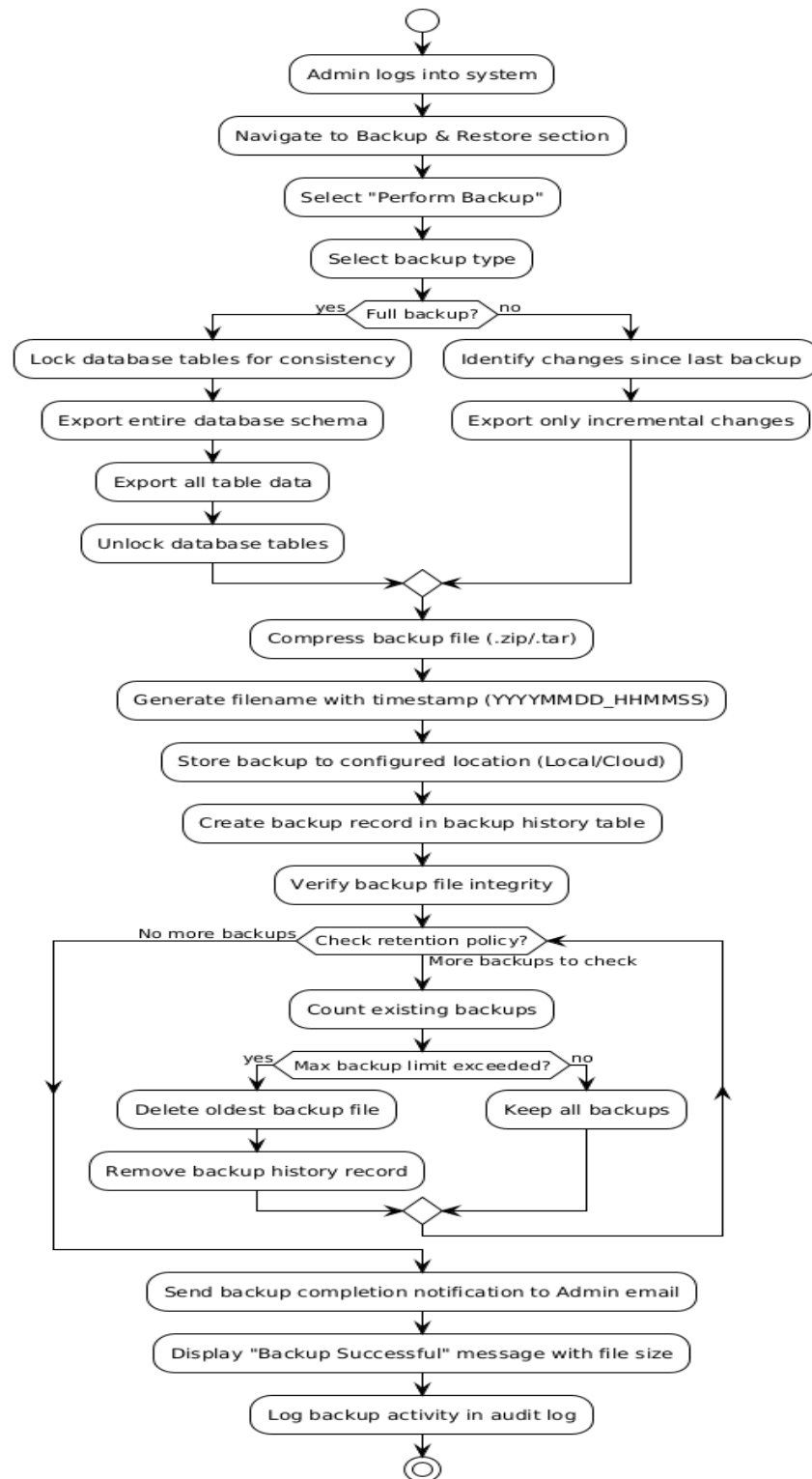
Fee Management System - Activity Diagram (Dues & Fine Management)



Activity Diagram 4: Defaulter Report Generation

Fee Management System - Activity Diagram (Defaulter Report Generation)

Activity Diagram 5: Database Backup Process

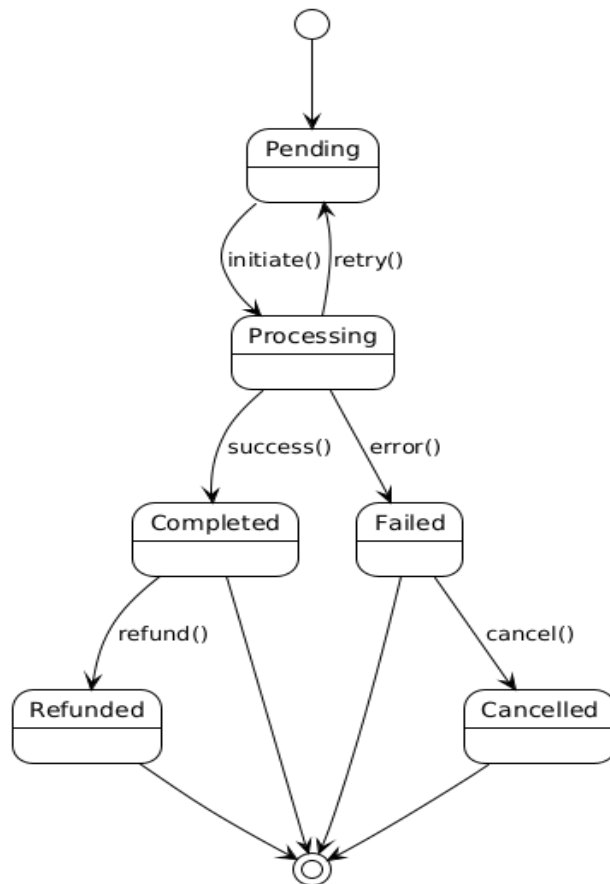
Fee Management System - Activity Diagram (Database Backup)

4.8. State Transition Diagram

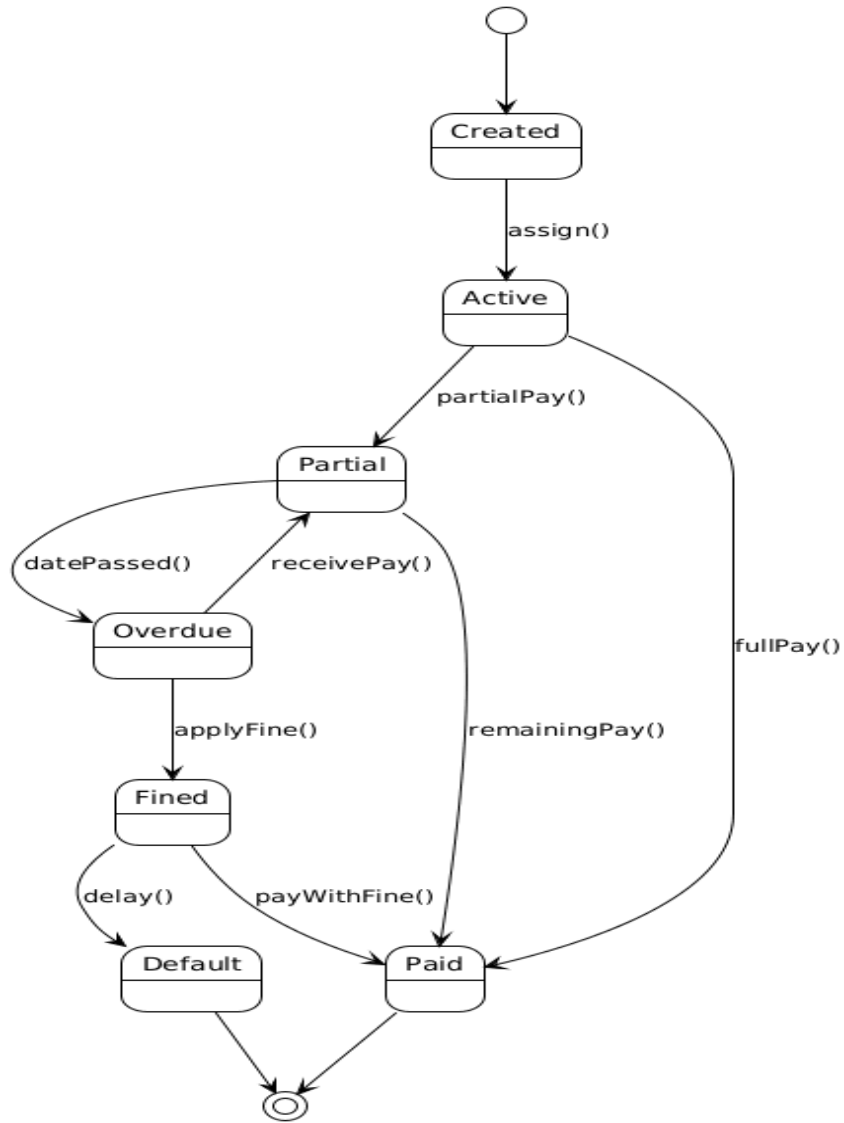
A state transition diagram is a UML behavioral diagram that shows the different states an object can be in during its lifecycle and the events that cause transitions between those states. It represents the dynamic behavior of a single object or system component by illustrating states, initial and final states, transitions, events, and actions. State transition diagrams are particularly useful for modeling objects that have complex lifecycle behavior, such as payments, dues, and student status

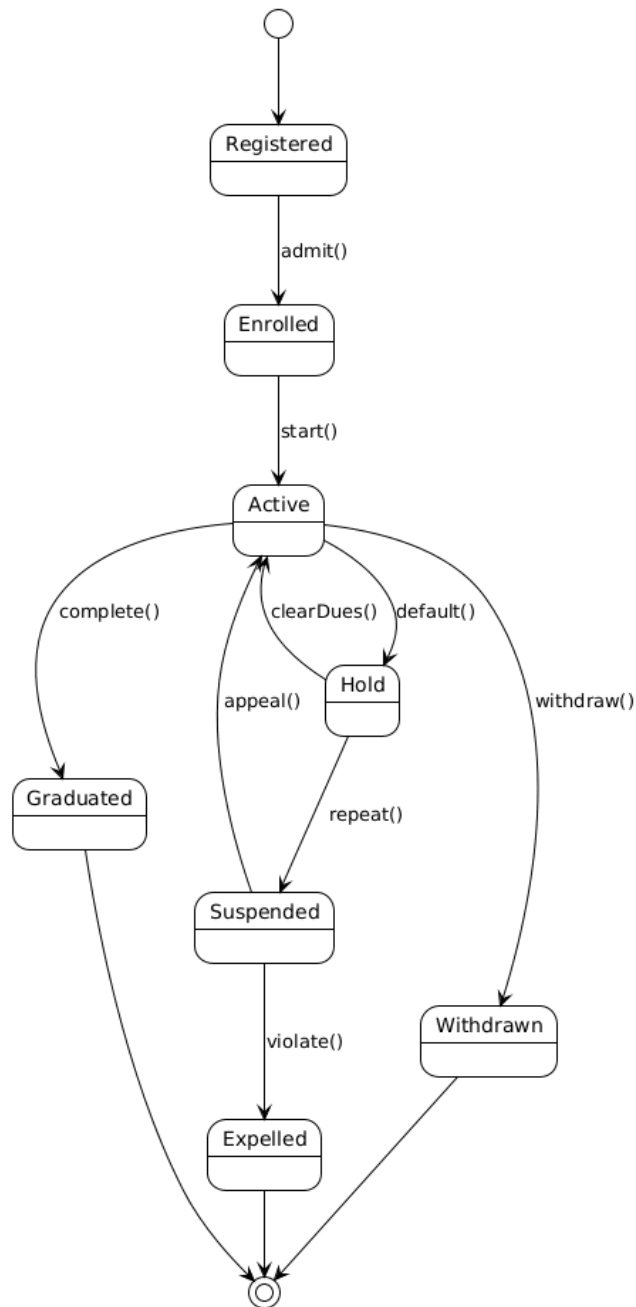
State Transition Diagram 1: Payment State

Figure 4.X: State Transition Diagram (Payment)

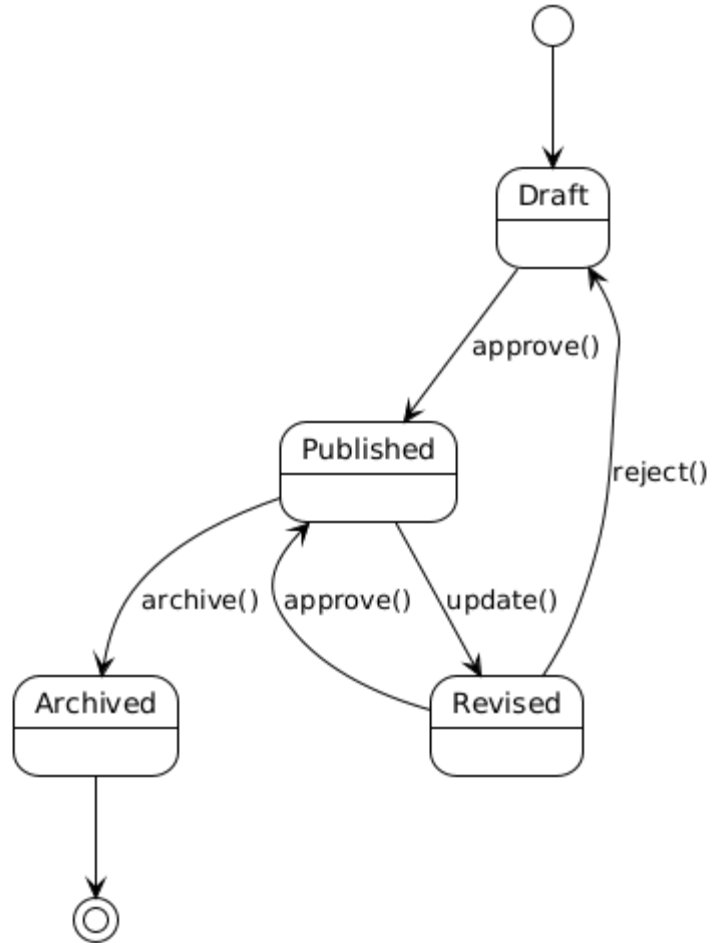


State Transition Diagram 2: Dues State

Figure 4.X: State Transition Diagram (Dues)

State Transition Diagram 3: Student Status**Figure 4.X: State Transition Diagram (Student Status)**

State Transition Diagram 4: Fee Structure

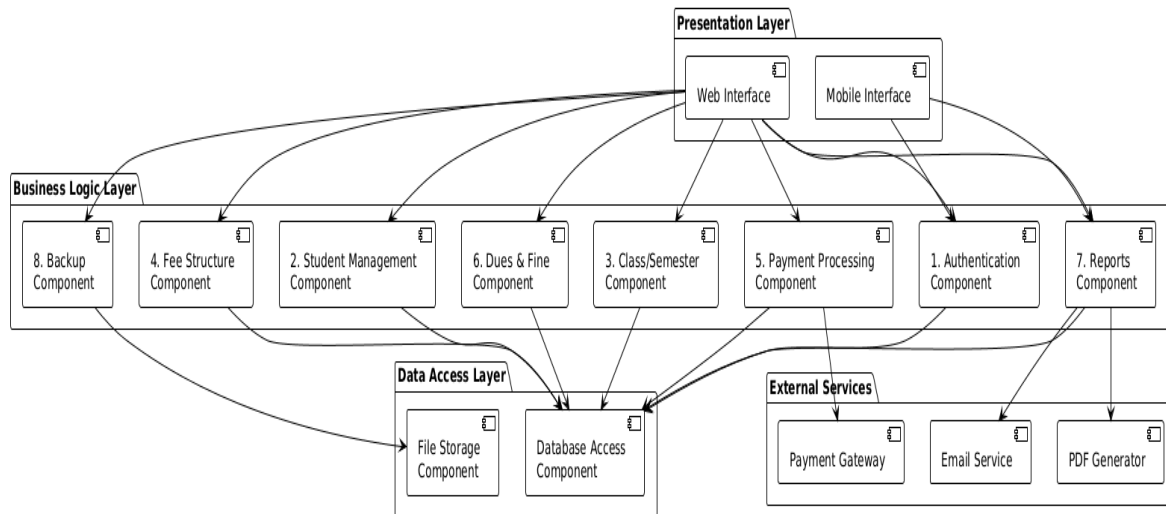
Figure 4.X: State Transition Diagram (Fee Structure)

4.9. Component Diagram

Component Diagram 1: System Component Diagram

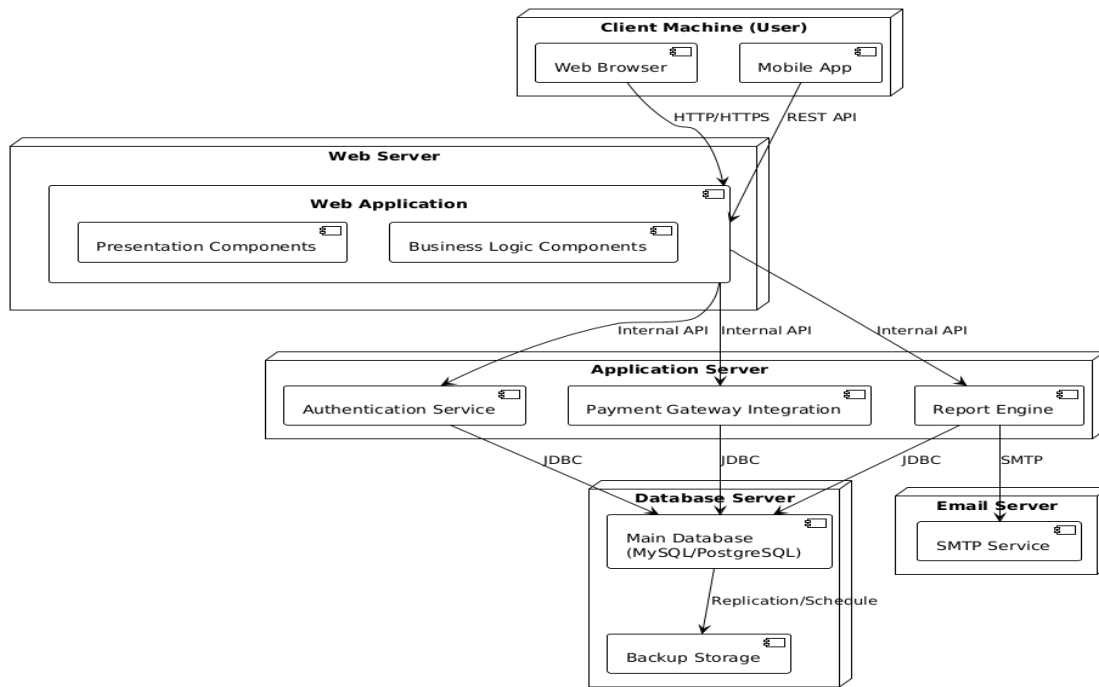
A system component diagram is a UML structural diagram that shows the organization and dependencies of software components within a system. It illustrates how components are grouped into layers (presentation, business logic, data access), how they interact through interfaces and dependencies, and how they communicate with external services. This diagram provides a high-level view of the system's modular architecture.

Figure 4.X: System Component Diagram (Fee Management System)



Component Diagram 2: Component Deployment Diagram

A component deployment diagram is a UML diagram that shows the physical distribution of software components across hardware nodes (servers, client machines, etc.). It illustrates where each component resides, the network protocols used for communication, and the physical relationships between hardware and software elements. This diagram helps visualize the system's deployment architecture and infrastructure requirements.

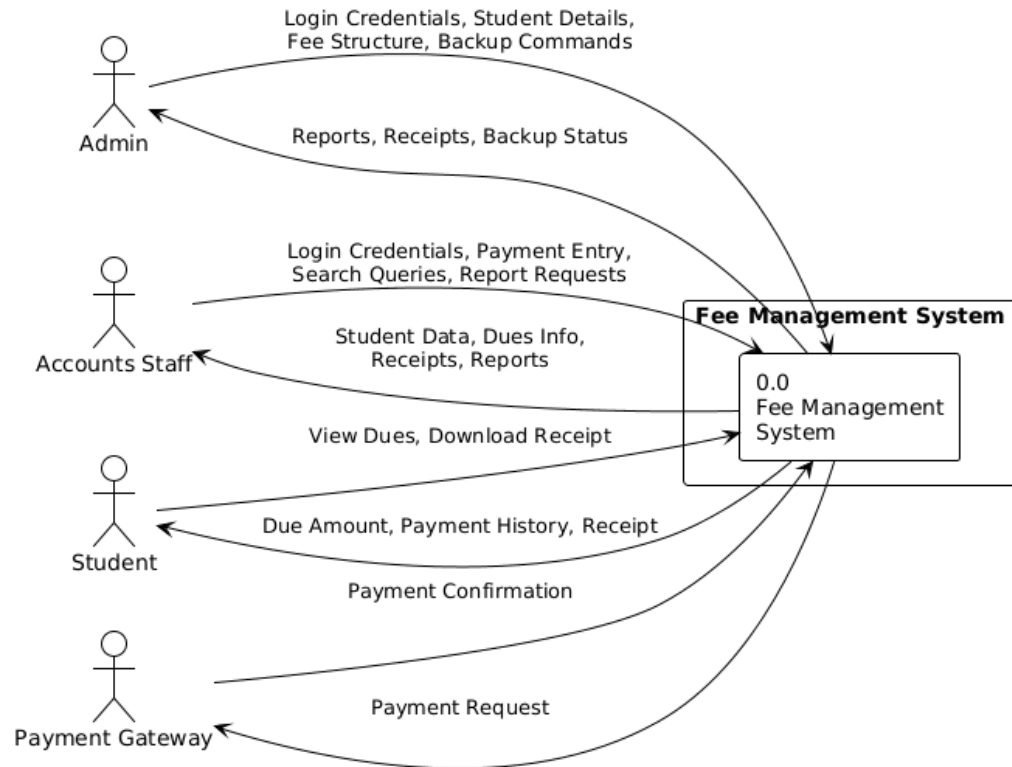
Figure 4.X: Component Deployment Diagram (Fee Management System)

4.10. Data Flow diagram [only if structured approach is used - Level 0 and 1]

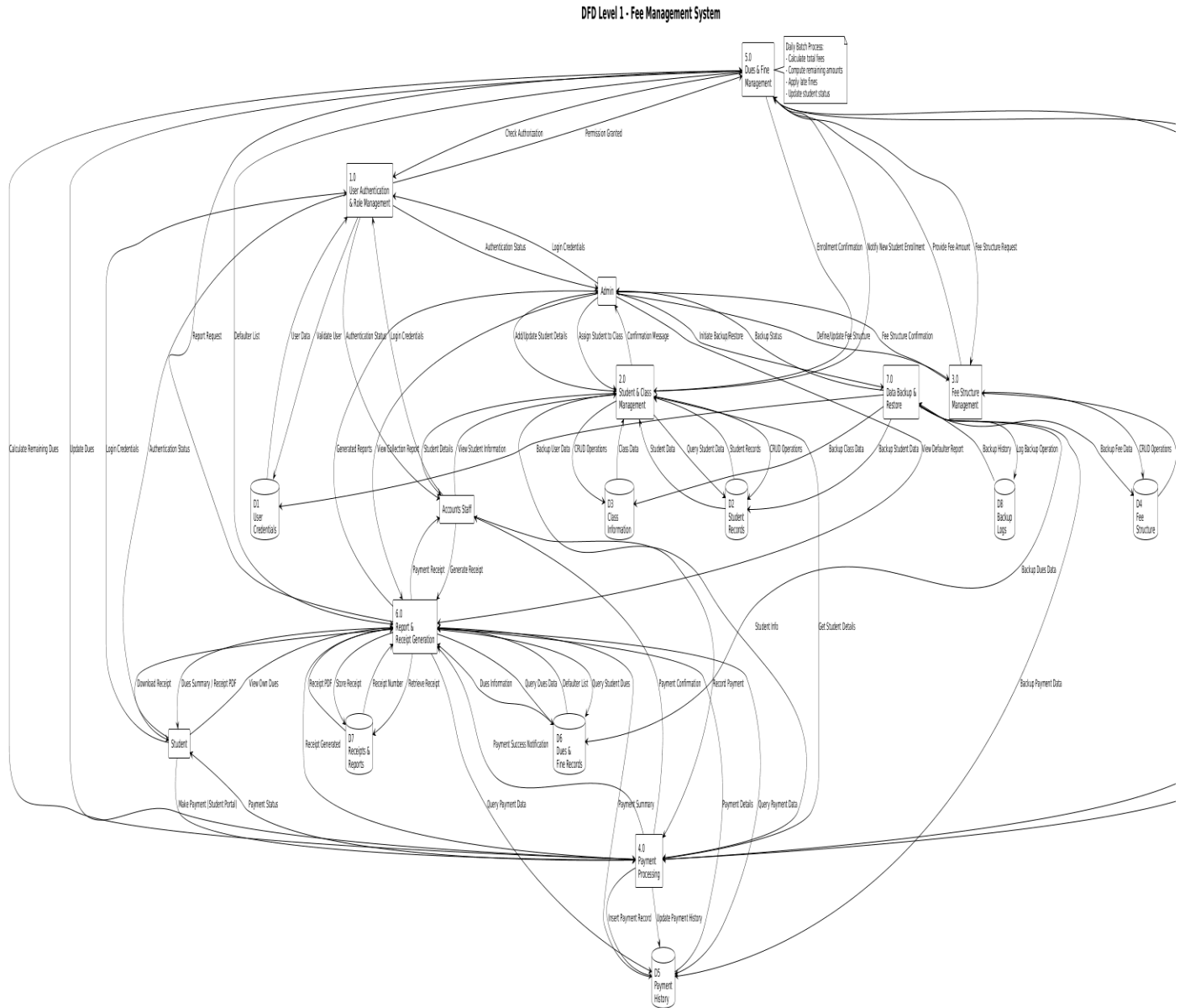
A Data Flow Diagram (DFD) is a graphical representation that shows the flow of data through a system, including processes, data stores, external entities, and data flows. It illustrates how data moves from external entities into the system, how it is processed, stored, and how output data is produced. DFDs are used to visualize system requirements and functional dependencies without showing any control flow or programming logic. This section includes Level 0 (Context Diagram) and Level 1 DFD for the Fee Management System.

Level 0 DFD (Context Diagram)

The Context Diagram shows the entire Fee Management System as a single process with all external entities that interact with the system

Figure 4.X: Level 0 DFD - Context Diagram (Fee Management System)**Level 1 DFD**

The Level 1 DFD breaks down the main process into major sub-processes showing detailed data flows and data stores.



Chapter 6

Testing and Evaluation

Chapter 6: Testing and Evaluation

This chapter presents the systematic testing and evaluation of the Fee Management System to ensure its functionality, reliability, and performance. It describes various testing techniques including use case testing, equivalence partitioning, boundary value analysis, data flow testing, unit testing, integration testing, performance testing, and stress testing. Each testing method is applied to validate different aspects of the system, from individual components to overall system behavior. The results of these tests ensure that the system meets all specified requirements and operates correctly under various conditions. This chapter serves as a quality assurance blueprint for the Fee Management System.

6.1. Use Case Testing

Use case testing is a black-box testing technique that validates the system against the scenarios described in use cases. It focuses on testing the flow of events, including basic flows (normal scenarios) and alternative flows (exceptional scenarios). This technique ensures that the system behaves correctly when users interact with it.

Test Case 1: Process Fee Payment (Basic Flow)

Element	Description
Use Case	Process Fee Payment
Test ID	UC-01
Description	Accounts Staff successfully processes full fee payment
Pre-conditions	Staff is logged in, Student exists, Dues pending
Test Steps	1. Search student by registration number 2. View pending dues 3. Select fee type 4. Enter full amount

Element	Description
	5. Select payment mode
	6. Confirm payment
Expected Result	Payment recorded, receipt generated, dues updated
Actual Result	
Status	

Test Case 2: Login with Invalid Credentials (Alternative Flow)

Element	Description
Use Case	User Authentication
Test ID	UC-02
Description	User attempts login with wrong password
Pre-conditions	User account exists
	1. Enter valid username
Test Steps	2. Enter wrong password
	3. Click login
Expected Result	Error message "Invalid Credentials" displayed
Actual Result	
Status	

6.2. Equivalence partitioning

Equivalence partitioning is a black-box testing technique that divides input data into valid and invalid partitions (equivalence classes). Test cases are designed to represent each partition, assuming that all values in a partition behave the same way.

Test Cases for Fee Amount Field:

Partition	Input Range	Test Value	Expected Result
Valid - Positive	1 - 100,000	5,000	Accepted
Invalid - Zero	0	0	Rejected with error
Invalid - Negative	-1 to $-\infty$	-500	Rejected with error
Invalid - Too high	> 100,000	200,000	Rejected with error

Test Cases for Student Age Field:

Partition	Input Range	Test Value	Expected Result
Valid	5 - 100	20	Accepted
Invalid - Below	< 5	3	Rejected with error
Invalid - Above	> 100	150	Rejected with error

Test Cases for Percentage/Score Field:

Partition	Input Range	Test Value	Expected Result
Valid	0 - 100	75	Accepted
Invalid - Below	< 0	-10	Rejected with error
Invalid - Above	> 100	120	Rejected with error

6.3. Boundary value analysis

Boundary value analysis is a testing technique that focuses on values at the boundaries (edges) of equivalence partitions. It tests minimum, maximum, and just inside/outside boundary values where errors are most likely to occur.

Boundary Tests for Fee Amount (Valid Range: 1 - 100,000):

Test ID	Value	Position	Expected Result
BVA-01	0	Just below minimum	Rejected
BVA-02	1	Minimum	Accepted

Test ID	Value	Position	Expected Result
BVA-03	2	Just above minimum	Accepted
BVA-04	99,999	Just below maximum	Accepted
BVA-05	100,000	Maximum	Accepted
BVA-06	100,001	Just above maximum	Rejected

Boundary Tests for Student Age (Valid Range: 5 - 100):

Test ID	Value	Position	Expected Result
BVA-07	4	Just below minimum	Rejected
BVA-08	5	Minimum	Accepted
BVA-09	6	Just above minimum	Accepted
BVA-10	99	Just below maximum	Accepted
BVA-11	100	Maximum	Accepted
BVA-12	101	Just above maximum	Rejected

Boundary Tests for Percentage (Valid Range: 0 - 100):

Test ID	Value	Position	Expected Result
BVA-13	-1	Just below minimum	Rejected
BVA-14	0	Minimum	Accepted
BVA-15	1	Just above minimum	Accepted
BVA-16	99	Just below maximum	Accepted
BVA-17	100	Maximum	Accepted
BVA-18	101	Just above maximum	Rejected

6.4. Data flow testing

Data flow testing is a white-box testing technique that focuses on the points where variables are defined, used, and undefined throughout the program. It traces the lifecycle of data variables to identify anomalies.

Data Flow Test for Payment Processing:

Variable	Define (D)	Use (U)	Kill/Undefine (K)	Test Case
amountPaid	User input	Calculate dues, update balance	After receipt generation	Enter valid amount
studentId	Search input	Fetch student record	After payment completion	Enter existing ID
paymentMode	Selection	Validate, process payment	After transaction	Select valid mode

Data Flow Anomalies Checked:

Anomaly	Meaning	Status
DU (Define-Use)	Variable defined then used	
UD (Use-Define)	Variable used before definition	
DK (Define-Kill)	Variable defined then killed without use	
UK (Use-Kill)	Variable used then killed	

Data Flow Test Cases:

Test ID	Variable	Test Steps	Expected Result
DFT-01	amountPaid	Define amount → Use in calculation → Kill after receipt	No data anomaly

Test ID	Variable	Test Steps	Expected Result
DFT-02	studentId	Define search ID → Use to fetch record	Valid record returned
DFT-03	transactionId	Generate after payment → Use in receipt → Kill after email	Transaction logged

6.5. Unit testing

Unit testing is a white-box testing technique where individual components or modules of the system are tested in isolation to verify they function correctly. Each unit is tested independently using specific test cases.

Unit Test Cases:

Test ID	Module	Test Description	Input	Expected Output	Status
UT-01	Authentication	Validate correct login	username="admin", pwd="1234"	Login Success = True	
UT-02	Authentication	Validate wrong password	username="admin", pwd="wrong"	Login Success = False	
UT-03	Authentication	Empty fields	username="", pwd=""	Validation Error	
UT-04	Student Add	Add new student	Valid student data	Student Saved, ID generated	
UT-05	Student Add	Duplicate registration	Same reg number	Error: Already exists	
UT-06	Fee Calculation	Calculate total fees	Fee list = [5000,2000,1000]	Total = 8000	

Test ID	Module	Test Description	Input	Expected Output	Status
UT-07	Dues Calculation	Calculate remaining	Total=10000, Paid=4000	Remaining=6000	
UT-08	Fine Calculation	Late fee for 5 days	Daily fine=10, Days=5	Fine=50	
UT-09	Receipt Generation	Generate receipt number	Payment ID=101	Unique receipt number	
UT-10	Database Connection	Test connection validity	Connection string	Connected = True	

6.6. Integration testing

Integration testing verifies that different modules of the system work together correctly. It focuses on interfaces and interactions between integrated components.

Test ID	Modules Integrated	Test Description	Expected Result	Status
IT-01	Authentication + Student Management	Login then add student	Only logged-in Admin can add student	
IT-02	Student Management + Class Management	Assign student to class	Student linked to class successfully	
IT-03	Fee Structure + Dues	Create fee structure, calculate dues	Dues calculated correctly	
IT-04	Payment + Dues	Process payment, update dues	Dues amount decreases by payment	

Test ID	Modules Integrated	Test Description	Expected Result	Status
IT-05	Payment + Receipt	Process payment, generate receipt	Receipt generated with payment details	
IT-06	Dues + Fine	Overdue dues, fine applied	Fine added to total dues	
IT-07	Student + Reports	Search student, generate report	Report includes student data	
IT-08	Backup + Database	Perform backup, verify data	Backup created and verified	
IT-09	Payment Gateway + Payment	External gateway integration	Payment confirmation received	
IT-10	All modules	End-to-end fee payment flow	Complete flow works correctly	

Integration Testing Approach:

Type	Description	Used In
Top-Down	Test high-level modules first	✓ Yes
Bottom-Up	Test low-level modules first	✓ Yes
Big Bang	Test all modules together	✓ Yes
Incremental	Add modules one by one	✓ Yes

6.7. Performance testing

Performance testing evaluates how the system performs under expected workload conditions. It measures response time, throughput, resource utilization, and stability.

Performance Test Scenarios:

Test ID	Scenario	Load	Response Time (Expected)	Response Time (Actual)	Status
PT-01	Login	10 concurrent users	< 3 seconds		
PT-02	Search Student	100 concurrent searches	< 2 seconds		
PT-03	Process Payment	20 concurrent payments	< 5 seconds		
PT-04	Generate Report	50 report requests	< 10 seconds		
PT-05	View Dashboard	30 concurrent views	< 3 seconds		
PT-06	Database Query	500 records retrieval	< 2 seconds		

Performance Metrics Measured:

Metric	Target	Result	Status
Average Response Time	< 3 seconds		
Peak Response Time	< 10 seconds		
Throughput (requests/sec)	> 50		
CPU Utilization	< 80%		
Memory Utilization	< 70%		
Database Query Time	< 2 seconds		

6.8. Stress Testing

Stress testing evaluates how the system behaves under extreme conditions, such as high load, limited resources, or beyond normal capacity. It identifies breaking points and recovery capabilities.

Stress Test Scenarios:

Test ID	Scenario	Load Condition	Duration	Expected Behavior	Actual Behavior	Status
ST-01	Sudden peak load	500 concurrent users	5 minutes	System handles without crash		
ST-02	Maximum students	10,000 student records	N/A	Search works within 5 sec		
ST-03	Bulk payment	1,000 payments simultaneously	10 minutes	All payments processed		
ST-04	Extended run	Normal load for 24 hours	24 hours	No memory leaks		
ST-05	Database overload	Complex queries with 20 joins	20 queries/sec	No timeout		
ST-06	Network latency	500ms delay simulated	30 minutes	Graceful degradation		

System Behavior under Stress:

Condition	System Response
80% load	Normal operation
100% load	Response time increases 2x
120% load	Queue mechanism activates

Condition	System Response
-----------	-----------------

150% load	System returns "Try again" message
-----------	------------------------------------

200% load	Graceful degradation, no crash
-----------	--------------------------------